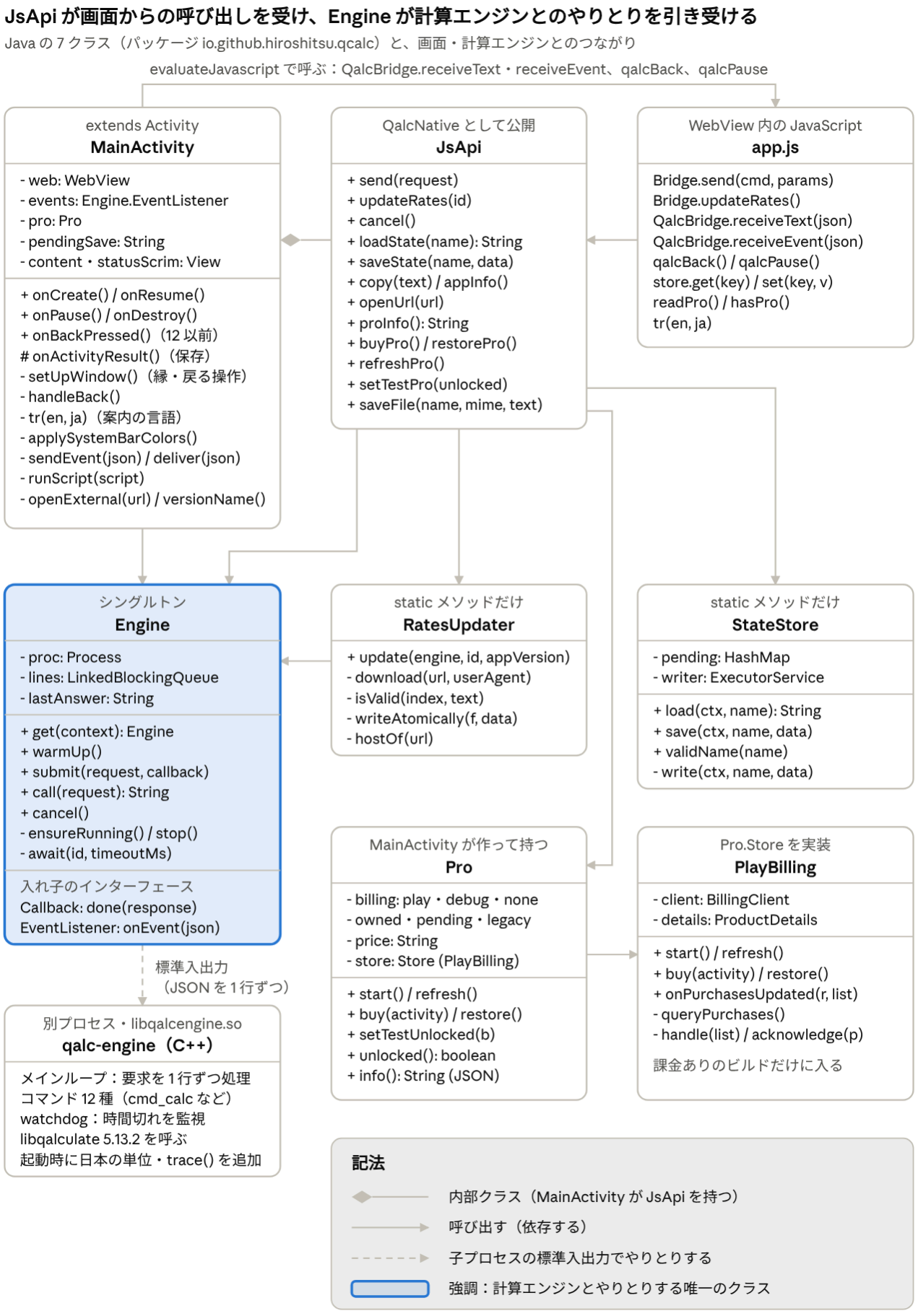
3. 詳細設計書

クラス、通信プロトコル、処理手順、データ形式、時間制限を、ソースコードの実装（版 1.3.0）どおりに記述します。画面の文は英語（標準）と日本語の 2 通りあり、この章では日本語の画面の文を「」で示します。エンジンと Java が返す文は英語なので、英語のまま示します（日本語の画面での表示は 3.14）。

3.1 クラス構成と責務


JsApi が画面からの呼び出しを受け、Engine が計算エンジンとのやりとりを引き受ける
Java の 7 クラス（パッケージ io.github.hiroshitsu.qcalc）と、画面・計算エンジンとのつながり
evaluateJavascript で呼ぶ：QalcBridge.receiveText・receiveEvent、qalcBack、qalcPause
extends Activity
MainActivity
- web: WebView
- events: Engine.EventListener
- pro: Pro
- pendingSave: String
- content・statusScrim: View
+ onCreate() / onResume()
+ onPause() / onDestroy()
+ onBackPressed()（12 以前）
onActivityResult()（保存）
- setUpWindow()（縁・戻る操作）
- handleBack()
- tr(en, ja)（案内の言語）
- applySystemBarColors()
- sendEvent(json) / deliver(json)
- runScript(script)
- openExternal(url) / versionName()
QalcNative として公開
JsApi
+ send(request)
+ updateRates(id)
+ cancel()
+ loadState(name): String
+ saveState(name, data)
+ copy(text) / appInfo()
+ openUrl(url)
+ proInfo(): String
+ buyPro() / restorePro()
+ refreshPro()
+ setTestPro(unlocked)
+ saveFile(name, mime, text)
WebView 内の JavaScript
app.js
Bridge.send(cmd, params)
Bridge.updateRates()
QalcBridge.receiveText(json)
QalcBridge.receiveEvent(json)
qalcBack() / qalcPause()
store.get(key) / set(key, v)
readPro() / hasPro()
tr(en, ja)
シングルトン
Engine
- proc: Process
- lines: LinkedBlockingQueue
- lastAnswer: String
+ get(context): Engine
+ warmUp()
+ submit(request, callback)
+ call(request): String
+ cancel()
- ensureRunning() / stop()
- await(id, timeoutMs)
入れ子のインターフェース
Callback: done(response)
EventListener: onEvent(json)
static メソッドだけ
RatesUpdater
+ update(engine, id, appVersion)
- download(url, userAgent)
- isValid(index, text)
- writeAtomically(f, data)
- hostOf(url)
static メソッドだけ
StateStore
- pending: HashMap
- writer: ExecutorService
+ load(ctx, name): String
+ save(ctx, name, data)
+ validName(name)
- write(ctx, name, data)
MainActivity が作って持つ
Pro
- billing: play・debug・none
- owned・pending・legacy
- price: String
- store: Store (PlayBilling)
+ start() / refresh()
+ buy(activity) / restore()
+ setTestUnlocked(b)
+ unlocked(): boolean
+ info(): String (JSON)
Pro.Store を実装
PlayBilling
- client: BillingClient
- details: ProductDetails
+ start() / refresh()
+ buy(activity) / restore()
+ onPurchasesUpdated(r, list)
- queryPurchases()
- handle(list) / acknowledge(p)
課金ありのビルドだけに入る
標準入出力
（JSON を 1 行ずつ）
別プロセス・libqalcengine.so
qalc-engine（C++）
メインループ：要求を 1 行ずつ処理
コマンド 12 種（cmd_calc など）
watchdog：時間切れを監視
libqalculate 5.13.2 を呼ぶ
起動時に日本の単位・trace() を追加
記法
内部クラス（MainActivity が JsApi を持つ）
呼び出す（依存する）
子プロセスの標準入出力でやりとりする
強調：計算エンジンとやりとりする唯一のクラス

クラス図 · Java の 7 クラスと画面・計算エンジン

Java 側のクラスは 7 つです（パッケージ io.github.hiroshitsu.qcalc。PlayBilling は Google Play Billing Library を入れてビルドしたときだけ含まれる。3.20）。画面からの呼び出しはすべて JsApi が受け、計算エンジンとのやりとりは Engine だけが行います。

クラス	ファイル（行数）	責務	動くスレッド
MainActivity	MainActivity.java（574）	WebView を作って設定する（JavaScript と DOM Storage は有効、ファイル・コンテンツへのアクセスは無効、文字の拡大率は 100 %）。応答とイベントを画面に渡す（evaluateJavascript）。ウィンドウの準備（setUpWindow：Android 15 以降の画面の縁、Android 13 以降の戻る操作の受け口。3.15）、戻る操作（handleBack）、一時停止（qalcPause で保存）、終了時の後片付け、表示処理が止まったときの作り直し、外部リンク（http・https だけ）、システムバーの色、アプリの版の取得（versionName。appInfo と為替レートの取得で使う）、QCalc Pro の準備（起動の最初に Pro を作って購入を問い合わせ、画面に戻るたびに問い合わせ直し、終了時に接続を閉じる。3.20）、ファイルの保存（システムのファイル選択画面を開き、選ばれた場所に qalc-save スレッドで書く。3.23）、Java が自分で出す文の言語の選択（tr。3.21）	UI スレッド

クラス	ファイル（行数）	責務	動くスレッド
JsApi	MainActivity.java の内部クラス	画面に QalcNative として公開する 14 のメソッド（3.2）。時間のかかる処理は Engine・StateStore・別スレッドに渡してすぐに戻る	WebView の JavaBridge スレッド
Engine	Engine.java（357）	計算エンジンの起動・監視・再起動。要求を送って同じ id の応答を待つ。時間切れ・中止・異常終了の判定。ans の保存と、新しいエンジンへの復元	作業スレッド 1 本（要求を 1 件ずつ処理）、標準出力の読み取りスレッド、標準エラーの読み捨てスレッド
StateStore	StateStore.java（123）	画面の状態（履歴・設定・入力中の式など）をすぐにファイルへ保存する（3.12）	書き込みスレッド 1 本
RatesUpdater	RatesUpdater.java（164）	為替レートのダウンロード、検査、ファイルの置き換え、エンジンへの再読み込みの指示（3.11）。取得元に送る User-Agent には、MainActivity から受け取ったアプリの版を入れる	呼び出しごとの qalc-rates スレッド
Pro	Pro.java（213）	QCalc Pro の状態（購入済み・支払い待ち・以前からの利用者・価格）を持ち、画面に知らせる（info、pro イベント）。課金の種類 billing（play・debug・none）を決め、PlayBilling を名前で読み込む（入っていないビルドでも動く）。状態は SharedPreferences「pro」に保存する（3.20）	UI スレッドと JavaBridge スレッド（状態は synchronized で守る。PlayBilling の結果はどのスレッドからでも届く）

クラス	ファイル（行数）	責務	動くスレッド
PlayBilling	PlayBilling.java（296。app/src-billing）	Google Play Billing Library 8 による購入。Google Play への接続、価格と購入の問い合わせ、購入画面、購入の承認（acknowledge）、知らせる文（英語）。Gradle で qcalc.billing=true にしたビルドだけに入る（3.20）	Billing Library のスレッド（購入画面を開くのは UI スレッド）

主な呼び出し関係

- MainActivity → Engine：起動時の warmUp（エンジンの先行起動）、イベントの受け取り先の登録と解除。
- JsApi → Engine：send が submit（作業スレッドで 1 件ずつ処理）、cancel がエンジンの停止。
- JsApi → RatesUpdater：updateRates が新しいスレッドで update を呼ぶ。
- RatesUpdater → Engine：call で rates_info（id −2）と rates_reload（id −3）を送る。
- Engine → qalc-engine：標準入力に要求を 1 行、標準出力から応答とイベントを 1 行ずつ。
- MainActivity → Pro：起動の最初に Pro を作って start（Google Play への接続と購入の問い合わせ）、画面に戻るたび（onResume）に refresh、終了時（onDestroy）に end。Pro → PlayBilling：Store インターフェース（start・refresh・buy・restore・end）。PlayBilling → Pro：Owner インターフェース（owned・price・message）。Pro → MainActivity：Listener（pro イベントを画面へ）。
- JsApi → Pro：proInfo・buyPro・restorePro・refreshPro・setTestPro。JsApi → MainActivity：saveFile がファイル選択画面を開き、onActivityResult が qalc-save スレッドでファイルを書いて saved イベントを送る。

画面（app.js）の構成は 3.7、計算エンジン（qalc-engine.cc）の処理は 3.3〜3.6 で説明します。

3.2 JavaScript ブリッジの仕様

画面から Java へは window.QalcNative の 14 メソッド、Java から画面へは window.QalcBridge の 2 メソッドと、画面側の 2 つのフックを使います。計算の応答は戻り値では返さず、要求番号（id）で対応付けて非同期に届けます。

画面 → Java（window.QalcNative、JavaBridge スレッドで実行）

メソッド	引数	戻り値	動作
send(request)	1行の JSON 要求	なし	Engine の作業スレッドで処理し、応答を QalcBridge.receiveText で返す
updateRates(id)	要求番号（整数）	なし	別スレッド「qalc-rates」で為替レートを更新し、結果を receiveText で返す
cancel()	なし	なし	計算中のエンジンプロセスを停止する
loadState(name)	保存名	文字列または null	state/保存名.json を読む。まだ書いていない最新の値があればそちらを返す
saveState(name, data)	保存名、JSON 文字列（null で削除）	なし	書き込み用スレッドで非同期に保存する
copy(text)	文字列	なし	クリップボードに入れる（ラベル「QCalc」）。Android 12 以前は「コピーしました」と表示する（13 以降は OS が表示）。クリップボードが受け取らない（Android は一度に約 1 MB まで）ときは例外を捕まえ、「長すぎてコピーできませんでした」と表示する（版 1.2.0）
appInfo()	なし	JSON 文字列 {version, android, device, dark}	版、Android の版、端末名、ダークモードかどうか
openUrl(url)	URL	なし	http・https のときだけ外部のブラウザで開く
proInfo()	なし	JSON 文字列 {billing, pro, legacy, price, state}	QCalc Pro の状態（3.20）。billing は play・debug・none、pro は Pro が使えるか、legacy は Pro が始まる前からの利用者として使えるか、price は Google Play が示す価格の文字（¥300 など）、state は支払いの手続き中なら pending

メソッド	引数	戻り値	動作
buyPro()	なし	なし	UI スレッドで Google Play の購入画面を開く（PlayBilling があるときだけ）。結果は pro イベントで届く
restorePro()	なし	なし	購入を問い合わせ直す。見つからなければ、pro イベントの message で No purchase of QCalc Pro was found を知らせる
refreshPro()	なし	なし	価格と購入を問い合わせ直す（Pro の画面を開いたとき）
setTestPro(unlocked)	真偽	なし	テスト版（billing が debug）だけ。Pro を使える・使えない状態を切り替えて保存し、pro イベントを送る。ほかのビルドでは何もしない
saveFile(name, mime, text)	ファイル名、種類、内容	なし	システムのファイル選択画面（ACTION_CREATE_DOCUMENT）で保存先を選んでもらい、内容を UTF-8 で書く。名前は英数字で始まり英数字・「.」「_」「-」だけの 100 文字まで、種類は text/csv だけ、内容は 16,777,216 文字（16 × 1024 × 1024）まで。結果は saved イベント（3.23）

Java → 画面（UI スレッドの evaluateJavascript）

呼び出し	内容
QalcBridge.receiveText(text)	応答 1 行。解析して、同じ id で待っている Promise を解決する。解析できないときは、最も古い待ちを失敗として終わらせる
QalcBridge.receiveEvent(text)	イベント 1 件（JSON）。event が ans（エンジンの ans イベント。履歴の項目に挿入用の文字列を付ける）、pro（QCalc Pro の状態が変わった。PRO を読み直し、message があれば通知で表示する。3.20）、saved（ファイルの保存の結果。ok・cancelled・error。3.23）の 3 種類
window.qalcBack()	戻る操作。画面で処理したら true（Java は 0.8 秒待って応答がなければアプリを背面へ）

呼び出し	内容
window.qalcPause()	アプリが背面に回るとき。履歴と入力中の式をすぐ保存する

画面側の待ちには最終タイマーがあります。計算は「2 × 制限時間 × 式の数 + 60 秒」と 120 秒の大きいほう、為替更新は 300 秒で、これを過ぎると「計算エンジンから応答がありません」で終わらせます。通常は Java 側の制限（制限時間 + 10 秒）が先に働きます。

3.3 計算エンジンとの通信プロトコル

アプリはエンジンの標準入力に 1 行 1 件の JSON 要求を書き、エンジンは標準出力に 1 行 1 件の JSON 応答を返します。要求は 1 件ずつ順に処理され、応答には要求と同じ id が付きます。

共通の形

- 要求：`{"id":数値または文字列, "cmd":"コマンド名", …}`。画面は 1 から順に番号を付け、Java は内部用に -1（ans の復元）、-2・-3（為替更新）を使う。
- 成功：`{"id":…, "ok":true, …}`。失敗：`{"id":…, "ok":false, "error":"メッセージ"}`。JSON として読めない要求や、後ろに余分な文字がある要求は `{"id":null, "ok":false, "error":"bad request"}`。
- エラーの文（error）とメッセージは英語（エンジンと Java の文は版 1.3.0 から英語。それまでは日本語）。日本語の画面では、画面が MESSAGES_JA で日本語にして表示する（3.14、3.21）。
- 応答のあとでエンジンが終了するときは `"restart":true` が付く。Java はプロセスを破棄し、次の要求で起動し直す。
- 確定（store:true）で ans が変わると、応答の直後にイベント行 `{"event":"ans", "id":その計算の id, "value":"ans を再現する式", "input":"挿入用の文字列"}` が続く。Java はこの行を応答として扱わず、value を保存して画面に転送する。
- libqalculate 自身が標準出力に書く文字は、起動時に標準エラーへ振り替えて応答に混ざらないようにしている。標準エラーは Java が読み捨てる。

コマンド一覧

コマンド	要求の項目	応答の項目	主なエラー
init	なし	engine（"libqalculate 5.13.2"）、rates_time（為替レートの時刻、UNIX 秒）、ans_defined（ans に値があるか）、defs_error（定義ファイルを退避したときだけ）	—
calc	expr、opts（3.4）、timeout（ms、100〜120000、既定 3000）、store（既定 false）、preview（既定は store の反対）、max_chars（結果の最大バイト数。100〜10000000、既定 200000）、tail_chars（max_chars を超えたときに末尾も返すバイト数。0〜1000000、既定 0。2 倍が max_chars 以下のときだけ使う。版 1.3.0）	result、parsed、approx、comparison、timeout、truncated、chars・digits（切ったときだけ。全体の文字数と数字の数）、tail（tail_chars を指定して切ったときだけ。末尾）、messages（type は error・warning・info）、deferred（プレビューで実行を見送ったとき）	The expression is nested too deeply
set_ans	expr	set（設定できたか）	expr missing
plot	exprs（文字列の配列、最大 12）、min・max（既定 "-10"・"10"）、steps（10〜4000、既定 300）、var（既定 "x"）、opts、timeout（既定 8000、全式で共有）	series（式ごとに {x:[…], y:[…]}、値のない点は null）、messages	exprs missing、too many expressions、invalid range
plot3d	exprs（文字列の配列、最大 4）、xmin・xmax・ymin・ymax（既定 "-5"・"5"）、nx・ny（格子の点の数。2〜121、既定 41）、opts、timeout（既定 15000、全式で共有）	x（nx 個）、y（ny 個）、series（式ごとに z：ny × nx 個を行ごとに並べた配列で値のない点は null、complete：最後まで計算できたか）、messages	exprs missing、too many expressions、invalid x range、invalid y range

コマンド	要求の項目	応答の項目	主なエラー
catalog	kind（functions・variables・units）	items（関数：name・title・category・description・syntax・local、変数：name・display・title・category・description・local・value、単位：name・abbr・display・title・category・description・currency・local・composite・insert）。単位の abbr は、一覧に表示して式に挿入する文字（km、km/h、kW·h など）で、エンジンが「式に書くと同じ単位として読める」ことを確かめてから返す（3.18）。composite は複合単位（km/h など）の印、insert=false は式には挿入できず変換先（to のあと）にだけ使える単位	unknown catalog kind
rates_info	なし	sources（index・url・file・time）、time	—
rates_reload	なし	loaded（読み込めたか）、time、messages	—
define_var	name、value、title（任意）	saved（ファイルに保存できたか）、messages	Enter a name and a value、The variable name contains characters that cannot be used、"名前" is a built-in name and cannot be used、Check the value (…)

コマンド	要求の項目	応答の項目	主なエラー
define_func	name、expr（引数は x・y・z）、title（任意）	saved、messages	上記（変数名の文を除く）に加え、The function name contains characters that cannot be used、"to" cannot be used in the expression of a function
delete_def	kind（variable・function）、name	saved	There is no user definition "名前"
user_defs	なし	variables（name・value・title・temporary）、functions（name・expr・title）	—

例（確定の要求、応答、ans イベント）

```
{"id":19,"cmd":"calc","expr":"7/3","opts":
{"approx":"auto","precision":10},"timeout":20000,"store":true,"max_chars":450
000,"tail_chars":20000}
{"id":19,"ok":true,"result":"2 + 1/3 ≈ 2.333333333","parsed":"7 /
3","approx":false,"comparison":false,"timeout":false,"truncated":false,"messa
ges":[]}
{"event":"ans","id":19,"value":"7/3","input":"7/3"}
```

例（3D グラフの要求と応答。格子 3 × 3、2 つ目の式は x < 0 で実数にならない）

```
{"id":7,"cmd":"plot3d","exprs":
["x*y","sqrt(x)"],"xmin":"-1","xmax":"1","ymin":"-1","ymax":"1","nx":3,"ny":3
}
{"id":7,"ok":true,"x":[-1,0,1],"y":[-1,0,1],"series":[{"z":
[1,0,-1,0,0,0,-1,0,1],"complete":true},{"z":
[null,0,1,null,0,1,null,0,1],"complete":true}],"messages":[]}
```

3.4 計算オプションの対応表

画面は要求ごとに設定を opts として送り、エンジンは libqalculate の既定値から毎回組み立て直します（make_opts）。構造体の一部の項目が初期化されない libqalculate の仕様に備え、必ず静的な既定値をコピーしてから変更します。

常に固定する設定（公式 qalc の既定に合わせる）

- 入力：カンマは区切り、ドットは小数点。暗黙の掛け算は制限しない。解釈は適応モード。未知の名前は記号にせずエラーにする（unknowns_enabled = false）。
- 計算：分母を 0 でないと仮定したときに警告する。通貨はローカル通貨（日本では JPY）に換算する。
- 表示：記号は Unicode（×、−）、割り算は「/」、論理は true・false・or・and。進数には 0x・0b・0o を付け、指数は大文字の E。誤差は入力に ± があれば ±、interval() があれば区間、それ以外は有効数字で示す。

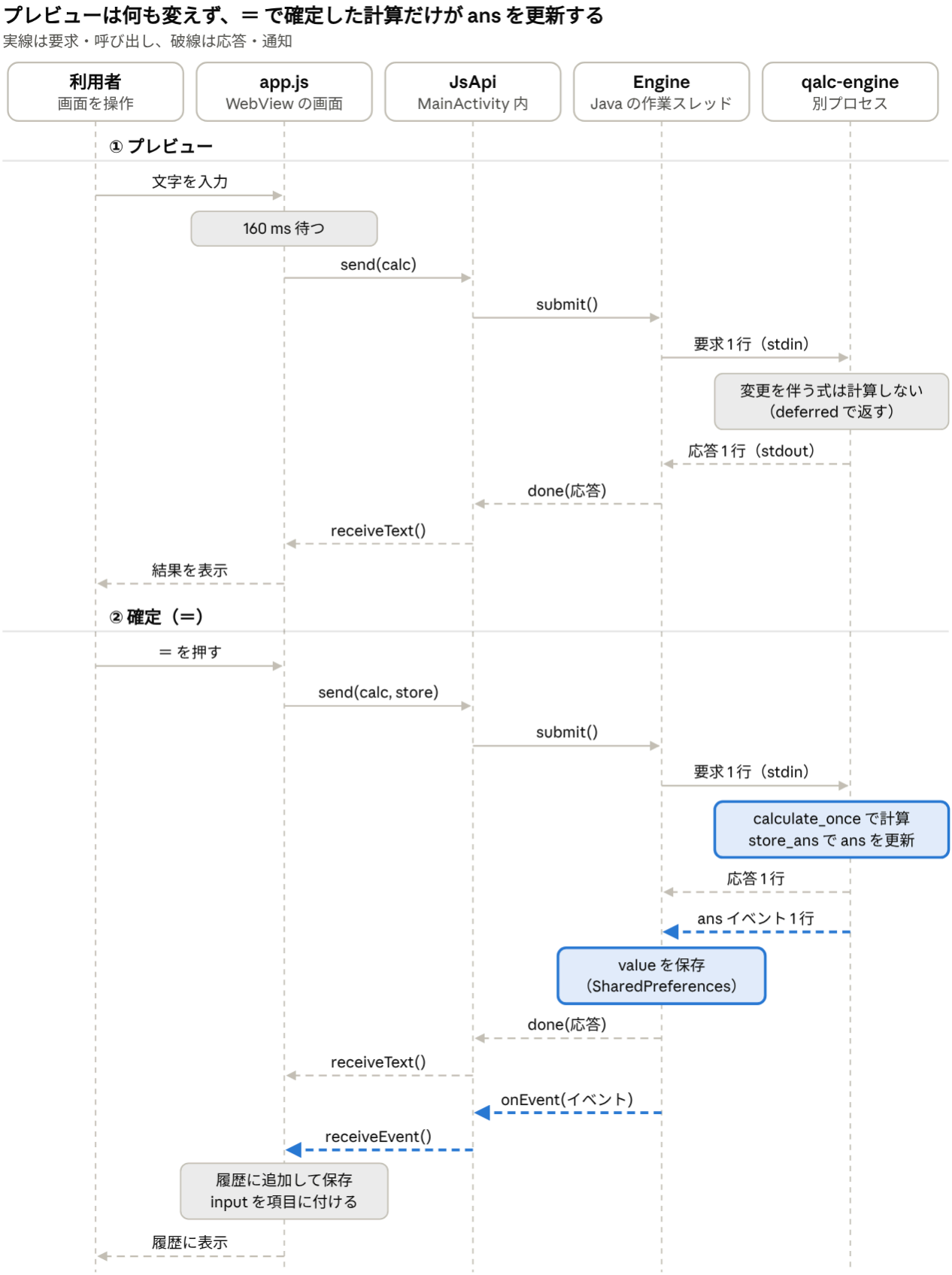
画面の設定との対応

opts の項目	値	libqalculate の設定	画面の設定項目
approx	auto（既定）・exact・approx	APPROXIMATION_TRY_EXACT + 自動近似 AUTO、APPROXIMATION_EXACT、APPROXIMATION_APPROXIMATE	厳密計算と近似（上部のボタンでも切替）
fraction	auto（既定）・decimal・exact・fraction・combined	自動分数 AUTO、FRACTION_DECIMAL、FRACTION_DECIMAL_EXACT、FRACTION_FRACTIONAL、FRACTION_COMBINED（後の 2 つは分数の長さを制限）	分数の表示
angle	rad（既定）・deg・gra	ANGLE_UNIT_RADIANS・DEGREES・GRADIANS	角度の単位（上部のボタンでも切替）
base	10（既定）・16・2・8・12・-1	結果の進数（-1 はローマ数字、2〜36 を受け付ける）	結果の進数（上部のボタンでは 10・16・2・8）

opts の項目	値	libqalculate の設定	画面の設定項目
precision	2〜200000（既定 10。版 1.1.0 までは 1000。QCalc Pro が使えないときは、画面が 1000 までに抑えて送る。3.20）	setPrecision（有効桁数）	有効桁数
max_decimals	-1（既定、制限なし）・0・2・4・6・8・10	use_max_decimals と max_decimals	小数点以下の最大桁数
notation	auto（既定）・sci・eng・pure・none	EXP_PRECISION、EXP_SCIENTIFIC、EXP_BASE_3、EXP_PURE、EXP_NONE	指数表記
units	optimal（既定）・si・base・none	POST_CONVERSION_OPTIMAL、OPTIMAL_SI、BASE、NONE	単位の自動変換
mixed_units	true（既定）・false	false のとき MIXED_UNITS_CONVERSION_NONE	複合単位で表示
complex	rect（既定）・exp・polar・cis	COMPLEX_NUMBER_FORM_RECTANGULAR・EXPONENTIAL・POLAR・CIS	複素数の形式
structuring	simplify（既定）・factorize・expand	STRUCTURING_SIMPLIFY・FACTORIZE・EXPAND	式の整理
grouping	false（既定）・true	true のとき DIGIT_GROUPING_STANDARD（エンジンは狭い空白で区切り、小数部も区切る。画面は整数部の区切りだけをカンマにする。3.9）	3桁ごとに区切る

approx と fraction の dual、入力の進数 input_base（2〜36）もエンジンは受け付けますが、画面には設定項目がありません。

3.5 計算処理の詳細



シーケンス図·入力中の表示（プレビュー）と確定

プレビューと確定はどちらも calc コマンドです。違うのは store（ans を更新するか）、preview（変更を伴う式を実行しないか）、時間制限と結果の長さの上限だけです。

calc の処理（cmd_calc）

1. timeout（100〜120000 ms、既定 3000）、max_chars、tail_chars、store（既定 false）、preview（既定は store の反対）を読み、opts から計算オプションを作る（3.4）。
2. 入れ子が深すぎる式は The expression is nested too deeply で断る（日本語の画面では「式の入れ子が深すぎます」）。
3. preview のときは、代入（x = 5、a := 1 など）や save() を含む式、および save・export・command・register・stack・plot の関数か、save・export・:= を含むユーザー関数を使う式を計算せず、deferred=true を返す。入力の途中で定義や保存が何度も実行されるのを防ぐため。
4. calculate_once で 1 回だけ評価する（次項）。
5. store で、値があり時間切れでなければ store_ans で ans を更新する。
6. 計算スレッドを強制的に打ち切った場合は応答に restart=true を付け、応答のあとでエンジンを終了する（次の要求で新しいエンジンが起動する）。
7. result が max_chars バイトを超えるときは、全体の文字数（chars）と数字の数（digits）を数える。tail_chars があり、その 2 倍が max_chars 以下なら、末尾の tail_chars バイトを tail として取り出し、先頭を max_chars − tail_chars バイトで切る（tail がなければ max_chars バイトで切る）。切った先頭には「…」を付け、どれも文字の途中では切らない。parsed は 2,000 バイトで切る。こうして応答を返す。ans が変わったときは、続けて ans イベントを出す。

1 回だけの評価（calculate_once）

libqalculate の calculateAndPrint と同じ手順を、式の評価を 1 回だけにして行います（乱数や a = a + 1 のような式を 2 回評価しないため）。評価した値をそのまま ans に使うので、表示した値と ans が必ず一致します。

1. コメント（# 以降）を取り除く。コメントだけの式は何もしない。
2. 「to」以降の変換先を分けて解釈する（進数、分数、因数分解、部分分数、暦、単位、2 進接頭辞など）。
3. 先頭の factor・factorize・expand を取り除き、それぞれの整理方法にする。
4. 「x = 5」のような代入を qalc と同じ規則で変数の保存に変える（transform_expression_for_equals_save）。
5. libqalculate の計算スレッドで、時間制限つきで評価する。時間内に終わらなければ timeout=true。スレッドを打ち切ったときは上の 6 のとおり。
6. 結果の整形は残り時間（最低 200 ms）の範囲で行う。単位がそのまま残った結果は最適な単位に変換する。近似と厳密の両方に意味があるときは、使った時間が制限の 1/3 未満なら厳密値も求め、「7/3 ≈ 2.333333333」のように並べる（print_dual）。ただし設定が

「自動」のときは、厳密な形に 6 項以上の和が含まれるか、全体の要素が 30 個を超えるなら、厳密な形は出さずに近似値だけを表示する（calculate_dual_exact の上限 5。qalc と同じ。版 1.1.0 で合わせた）。

7. 誤差つきの値（± や uncertainty()）は ± で、interval() は区間で、それ以外は有効桁で表示する。
8. approx（結果が近似値か）、comparison（比較・方程式の結果か）、parsed（解釈した式）を求める。時間切れのときは result を空にする。

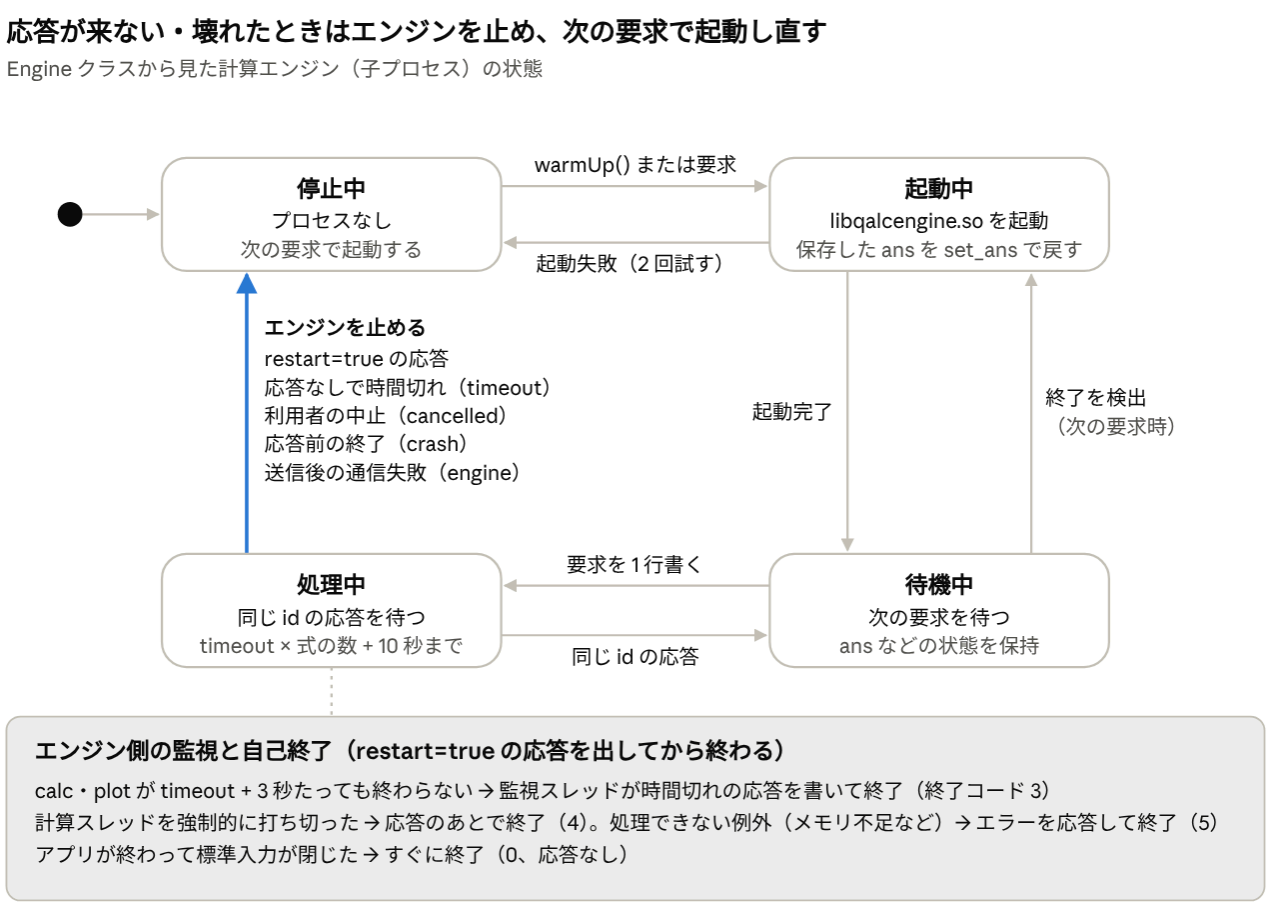
メッセージの回収（collect_messages）

計算で出た警告とエラーは、応答を作る直前に libqalculate のメッセージ一覧から取り出して messages に入れ、一覧を空にします。取り出す前に、開いたままの「メッセージの保留」があれば閉じて、中にたまっていたメッセージも一覧に加えます。libqalculate 5.13.2 が保留を閉じ忘れる問題への対処です（3.19）。

ans の規則（store_ans）

- 確定（store=true）で値があり、時間切れでないときだけ更新する。プレビューでは変わらない（ans を使った計算はできる）。
- 新しい値を入れる前に ans5 ← ans4 ← ans3 ← ans2 ← ans とずらす（qalc と同じ）。
- 方程式の解は解の値を入れる。x = 2 なら 2、x = 2 or x = −2 なら [2 −2]。解に and で条件が付くときは等式の右辺を使う。それ以外は結果そのもの。
- 更新したら応答の直後に ans イベントを出す。value は libqalculate の保存形式の文字列（新しいエンジンで ans を戻すため）、input は式に入れ直せる文字列（10 進、割り切れないときは分数。桁区切りなし、厳密な値は全桁）。
- Java は value を保存し（500,000 文字まで）、エンジンを起動し直すたびに set_ans（id −1）で ans に戻す。set_ans は、値の中でいちばん長い数字の並びの桁数 + 10 まで精度を一時的に上げて読む（上限 200,010。新しいエンジンは精度 10 で始まるので、上げないと近似値が約 38 桁に丸まる）。2,000 桁以上の数は libqalculate に渡さず GMP で直接読み、一時的な変数（qcalc_long_number_0 など）として式に入れて、計算のあとで消す。libqalculate は数を 1 桁ずつ読むため時間が桁数の 2 乗で増え、20 万桁の値で約 3.4 秒かかっていたが、GMP なら約 0.2 秒で、結果は同じ（テスト仕様書 6.2 の H15）。制限時間は 15 秒。戻すのは ans だけで、ans2〜ans5 は戻らない。
- 起動時にエンジンが ans_defined=false を返し、履歴があるときは、画面が最後の履歴の挿入用の文字列で set_ans を送る（以前の版からの引き継ぎ）。

3.6 エンジンプロセスの状態遷移と監視



状態遷移図 · 計算エンジンの 4 つの状態と復帰

Engine は計算エンジンを 1 つだけ動かし、要求を 1 件ずつ処理します（作業スレッドが 1 本で、call は synchronized）。中止の cancel だけはロックを取らずにプロセスを止めます（待っている call がロックを持っているため）。

応答の待ち方（await）

- 標準出力の読み取りスレッドが行をキューに入れる。ans イベントの行はキューに入れず、その場で保存して画面に知らせる。エンジンが終了したら終わりの印（EOF）を入れる。
- 待つ側は `{"id":同じ id,` で始まり `}` で終わる行だけを応答とみなし、ほかの行（前の要求の遅れた応答、途中で切れた行など）は読み飛ばす。
- 待ち時間は timeout × 式の数 + 10 秒（timeout がなければ 30 秒）。

復帰の規則

状況	Java の動き	画面への応答
応答に restart=true が付いている	応答をそのまま返し、プロセスを止める（次の要求で起動）	エンジンの応答

状況	Java の動き	画面への応答
待ち時間を過ぎた	プロセスを止める	code=timeout
利用者が中止した	cancel がプロセスを止め、待っていた要求が終わる	code=cancelled
送ったあとでエンジンが終了した（EOF）	止める。同じ要求は送り直さない（同じ式でまた終了する可能性が高いため）	code=crash
送ったあとの読み書きに失敗した	止める	code=engine
送る前に書き込みに失敗した（エンジンがすでに終了していた）	起動し直して 1 回だけ送り直す	2 回目も失敗なら code=engine
待機中にエンジンが終了していた	次の要求のときに終了を見つけ、起動し直してから送る	通常どおり

起動し直したエンジンには、保存してある ans を set_ans（id −1、20 秒まで待つ）で戻します。ユーザー定義はファイルから読み直されるので、起動し直しても利用者からは変わりません。ただし ans2〜ans5 と一時的な変数（x = 5 など）は消えます。

エンジン側の監視（qalc-engine.cc）

- calc・plot・plot3d を始めるときに、期限（timeout + 3 秒）と要求の id を監視スレッドに渡す。監視スレッドは 0.2 秒ごとに期限を確かめ、過ぎていたら timeout=true、restart=true の応答を書いて終了する（終了コード 3）。応答を書くと期限は解除される。
- libqalculate 自身の打ち切り（abort）は、約 5 秒 + 精度の立方根の秒数だけ待ってから計算スレッドを打ち切る。それより早く応答してエンジンを作り直すほうが、早く操作に戻れる。
- 計算スレッドを強制的に打ち切ったあとは libqalculate の状態が安全でないため、応答に restart=true を付けて終了する（4）。
- 処理中に例外（巨大な結果での std::bad_alloc など）が起きたら「engine error: …」と restart=true を返して終了する（4）。計算スレッドなどで捕まえられない例外は、on_terminate が Not enough memory for the calculation（日本語の画面では「計算に必要なメモリが不足しました」）と restart=true を書いて終了する（5）。
- 応答とログが混ざらないように、起動時に標準出力を複製して応答専用にし、元の標準出力（libqalculate が直接書くことがある）を標準エラーにつなぎ替える。Java は標準エラーを読み捨てる。
- 応答は 1 行ずつロックして書くので、監視スレッドの応答と通常の応答が混ざらない。
- 標準入力が閉じたら（アプリのプロセスがなくなった）、計算スレッドを待たずにすぐ終了する（0）。

エンジンの起動時の処理（main）

1. libqalculate の Calculator を作り、設定と定義の保存先（環境変数 QALCULATE_USER_DIR の下）を作る。
2. 為替レート（loadExchangeRates）と組み込みの定義（loadGlobalDefinitions）を読み込む。
3. 日本の単位 10 個（add_japanese_units）と trace()（add_matrix_functions）を追加する（3.19）。
4. ユーザー定義のファイルを確かめ（読めないものは退避する。3.12）、読み込む（loadLocalDefinitions）。ここまでに出たメッセージは捨てる。
5. 環境変数 QALC_CURRENCY（Java が端末の地域から決めた通貨。日本では JPY）が通貨の名前なら、ローカル通貨にする。
6. ans・ans2〜ans5 の変数を作る（最初は未定義）。
7. 標準出力を応答専用に複製して元の標準出力を標準エラーへつなぎ替え、監視スレッドを始めて、要求を待つ。

要求を 1 件受け取るたびに、命令を実行する前に、開いたままのメッセージの保留を閉じます（前の命令の残りを次に持ち越さない。中身は捨てる。3.19）。

3.7 画面処理（app.js）のモジュール構成

app.js は 1 つの即時実行関数の中に 23 の区画を並べた構成（2,653 行）で、外部ライブラリは使いません。行番号は版 1.3.0 のものです。画面の文は、英語と日本語の組から tr(英語, 日本語) で選びます（3.21）。ライセンス文は別ファイル licenses.js にあり、必要になったときだけ読み込みます（3.16）。

行	区画	主な関数・データ	役割
7–84	共通	el、$、store、LANG・JA・tr、debounce、vibrate、own、toast、hideToast、copyText、openUrl	要素の生成、保存（ファイルと localStorage の両方）、表示の言語と文の選択（3.21）、短い通知（「元に戻す」などのボタン付きも）
85–143	ブリッジ	Bridge.send、updateRates、backstop、receive、receiveText、receiveEvent	要求番号の発行、待ちの管理（Map）、最終タイマー、イベント（ans・pro・saved）の受け渡し
144–161	設定	DEFAULTS、TEXT_SCALE、textScale、applyTextSize、S、saveSettings	19 項目の既定値、文字の大きさ、設定の読み込みと保存

行	区画	主な関数・データ	役割
162–199	Pro	MAX_PRECISION・FREE_MAX_PRECISION、PRO、readPro、proOffered、hasPro、maxPrecision、engineOpts、上部のボタンの表示	QCalc Pro の状態（3.20）、有効桁数の上限、エンジンに渡す opts の組み立て
200–239	履歴データと要素	MAX_RESULT・TAIL_RESULT など、history、fitHistory（履歴の容量）、saveHistory、clip（長い文字の先頭）、exprEl、histPinned など	履歴の読み込みと保存（0.3 秒まとめて）、容量を超えた古い長い結果の短縮、最新の履歴を見せ続ける（ResizeObserver）
240–389	キー	PAGES（4 種類 × 25 個）、PAGE_TABS、TOOLBAR、KEY_NAMES、makeKey、bindPress、buildKeypad、setPage、renderPage、updateToolbar	キーとタブの定義、タブの切り替え、押下と繰り返し、読み上げ用の名前（英語・日本語）
390–570	式の編集	offerUndo、dropUndo、setExpr、insert、action、toggleKeyboard、autosize、normalizeJa、normalizeField	挿入（ひな形、選択範囲の囲み）、削除、消した式を元に戻す、カーソル移動（選択範囲と絵文字を考慮）、キーボードモード、全角文字などの変換、Enter
571–790	プレビューと行列の表示	exprChanged、saveDraft、runPreview、displayText、shownText、displayResult、splitMatrices、showResult、MESSAGES_JA（57 個）、localMessage、showPreview、cutMessage	入力中の計算、結果の表示形式（桁区切り、長い結果の先頭と末尾）、行列を表の形にする、メッセージの日本語化
791–884	確定	commit、setInput、イベントの受け取り（ans・pro・saved）、announce	確定、履歴への追加、挿入テキストの紐付け、中止
885–1033	履歴	insertText、wrapValue、legacyInsertText、insertResult、renderHistory、fitResultBox、fitMatrices、bindLongPress、historyMenu、historyChanged、tapButton	履歴の表示（空のときは最初の画面）、タップと長押し、結果の再利用、削除を元に戻す

行	区画	主な関数・データ	役割
1034–1131	シートとメニュー	sheet、closeSheet、confirmSheet、MENU（13 項目）、buildDrawer、menuAction	選択シート、確認、メニュー（Pro を買えないビルドでは QCalc Pro を出さない。Pro の機能に PRO の印）、履歴の消去と元に戻す
1132–1161	パネル	panel、closePanel、panelStack	パネルの生成（初回だけ）と重なりの管理
1162–1297	式の例	STARTERS（4 個）、EXAMPLES（12 分野・69 個）、TIPS（11 個）、useExample、openExamples	式の例と使い方のヒント。タップした例を入力欄に入れる（3.18）
1298–1541	一覧	loadCatalog、HIDDEN_ITEMS、CATEGORY_JA、JA_NAMES（124）、FUNC_JA（230）、VAR_JA（61）、JA_UNIT_TEXT、openCatalog、catalogRow、useItem	一覧の取得、日本語の名前と検索（日本語の画面だけ）、分類を指定して開く、挿入（3.18）
1542–1836	グラフ（2D）	openPlot、compute、autoView、niceStep、gridValues、viewOk、draw、zoom、refetchIfNeeded、setupPlotGestures、showPoint	2D と 3D の切り替え（3D は QCalc Pro）、点列の取得、canvas への描画、操作
1837–2141	3D グラフ	Q3、VIEW3D、viridis、build3d、zRange3d、compute3d、draw3d、pick3d、setup3dGestures、zoom3d	格子の値の取得、z の範囲、投影と陰影、奥から順の描画、回転・拡大・座標の表示（3.10）
2142–2204	ユーザー定義	openDefs、invalidateCatalogs	一覧、追加、削除
2205–2242	為替レート	openRates、fmtTime	手動更新
2243–2355	設定	THEMES（7 色）、SETTINGS（19 項目）、openSettings、settingsChanged、updateChips、cycle	設定画面（Pro の項目の確認、言語の切り替え）と上部の切替ボタン
2356–2424	このアプリについて	openAbout、PRIVACY_URL、SOURCE_URL、loadLicenses、openLicenses	版、式の例へのリンク、プライバシー、ライセンスの一覧と全文（3.16）

行	区画	主な関数・データ	役割
2425–2498	QCalc Pro	proPrompt、PRO_FEATURES、openPro、fillPro、buyPro、restorePro、proChanged	Pro の案内、Pro の画面（購入・復元・テスト版の切り替え）、状態が変わったときの反映（3.20）
2499–2543	履歴の書き出し	historyCsv、plainNumber、exportHistory、fileSaved	CSV の作成と保存（3.23）
2544–2653	戻る・中断・起動	qalcBack、qalcPause、applyTheme、autoUpdateRates、localizeStatic、chooseLanguage、setLanguage、boot、startEngine	戻る操作、背面への移行、配色テーマ（3.22）、言語（3.21）、起動処理、為替の自動更新

3.8 入力処理の詳細

入力欄は 1 つの textarea で、画面のキーでは端末のキーボードを出さず（inputmode="none"）、⌨ モードでは普通の文字入力に切り替えます。どちらの入力も最後は同じ「変更された」処理（exprChanged）に入ります。

キーの定義とタブ（PAGES、PAGE_TABS、setPage）

- PAGES は、タブの id（num・fn・calc・mat）ごとに 25 個のキーの定義を持つ。1 個の定義は、キーに出す文字と、入れる文字 ins、入れたあとにカーソルを戻す文字数 back、演算子の印 op（空欄のときに ans を前に付ける）、動作 act（del・clear・eq・more・kbd・left・right）、押し続けたときの繰り返し repeat、下付き・上付きを含む表示 html（$\int_a^b$、A^T、I_n）などの組。キーの並びは基本設計書 2.5。
- PAGE_TABS は 4 つのタブの「id、表示（123・Functions・Calculus・Matrix。日本語の画面では 123・関数・解析・行列）、読み上げ用の名前、関数の一覧の分類」。分類は解析が Calculus、行列が Matrices & Vectors で、「ほか…」キーが使う。
- buildKeypad が、いつも出ているキー 8 個（TOOLBAR）とタブ 4 個（#tab-num・#tab-fn・#tab-calc・#tab-mat、role="tab"）を作る。setPage(id) は 5×5 の領域（#grid）を選んだタブのキーに作り直し、選んだタブに on クラスと aria-selected="true" を付ける。選んだタブは保存しない（起動時は「123」）。
- 「ほか…」（act が more）は、いまのタブの分類を指定して関数の一覧を開く（openCatalog('functions', 分類)。3.18）。

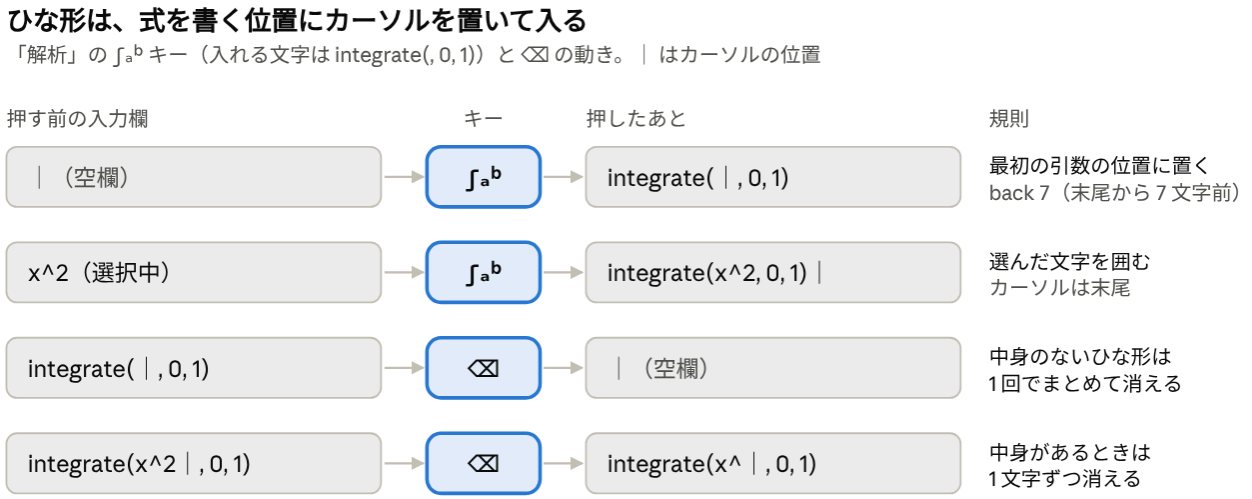
キーの押下（bindPress）

- pointerdown で既定の動作を止め、入力欄のフォーカスとカーソルを保つ。pointerup で実行する。

- ⌫・◀・▶は押し続けると 420 ms 後から 70 ms ごとに繰り返す。指がキーから外れるか、タッチが取り消されると止める。
- TalkBack やキーボードからの普通の click でも実行する。タッチのあとに続く click は無視し、二重に入力しない。

挿入と削除（insert、action）

- 文字はカーソル位置に入れる。キーの定義の back は「入れた文字の末尾から何文字戻った位置にカーソルを置くか」で、sin() は 1（括弧の中）、integrate(, 0, 1) は 7（最初の引数の位置）。選択範囲があって back が 1 以上のキーでは、選んだ文字をその位置に入れて囲み（x^2 を選んで ∫ₐᵇ を押すと integrate(x^2, 0, 1)）、カーソルは入れた文字の末尾に置く。
- 入力欄が空、履歴がある、設定「空欄で演算子を押すと ans から続ける」がオン、の 3 つを満たすとき、演算子キーは前に「ans」を付ける。端末のキーボードで * / ^ × ÷ ·（全角も可）を 1 文字だけ入れたときも同じ。
- ⌫ は選択範囲があればそれを消す。カーソルの前が「名前(」または「√(」「∛(」「∜(」で、後ろが「)」か、「, 0, 1)」のように括弧・区切りを含まない引数だけが続いて「)」で終わるなら、中身のない関数やひな形をまとめて消す（sin(｜) も integrate(｜, 0, 1) も 1 回で消える。dsolve(diff(y, x) = ｜) は 1 文字ずつ）。絵文字など 2 単位の文字はまとめて消す。
- ◀▶ は、選択範囲があればカーソルをその先頭（◀）・末尾（▶）に移して選択を解く。なければ 1 文字動かし、絵文字など 2 単位の文字（サロゲートペア）はまとめて動かす。



ひな形の挿入と削除 · ∫ₐᵇ キーと ⌫ の 4 つの場合

消した式を元に戻す（offerUndo、dropUndo、setExpr）

- AC で消した式と、履歴の式のタップ・「式を編集」・式の例のタップで入れ替わった式（setExpr）を 1 つだけ覚え（undoText）、入力欄の右上に「↶ 元に戻す」ボタン（#undo）を出す。空白だけの式は覚えない。
- ボタンを押すと、覚えていた式を入力欄に戻し、カーソルを末尾に置く。

- キーでの挿入、⌫ での削除、端末のキーボードからの入力、確定のどれかがあると、覚えていた式を捨ててボタンを隠す（新しい入力を上書きしないため）。
- 履歴の「この行を削除」と「履歴を消去」を元に戻すしくみは別で、通知のボタンを使う（3.9）。

確定のキー操作

- Enter の keydown（日本語変換中でないもの）、改行の beforeinput（insertLineBreak・insertParagraph）で確定する。
- それでも改行が入ったときは改行を空白に置き換え、末尾が改行なら確定する。

日本語入力の変換（normalizeJa）

どの置き換えも 1 文字を 1 文字に変えるので、カーソル位置はそのまま使えます。変換は日本語の変換中には行わず、変換が終わったとき（compositionend）に行います。プレビュー・確定・グラフ・ユーザー定義でも、送る前に同じ変換をかけます。

入力	変換後	例
全角の英数字・記号（U+FF01〜U+FF5E）	半角	１０＋５ → 10+5
全角の空白（U+3000）	半角の空白	—
円、￥	¥（日本円として読まれる）	100円 → 100¥
ハイフン類（U+2010・U+2011・U+2013〜U+2015）	-	—
数字・括弧・空白の間にある「ー」	-	５ー３ → 5-3（コーヒーなどの名前はそのまま）

キーボード入力モード（toggleKeyboard）

- ⌨ で inputmode を text と none に切り替え、text のときは body に kbd クラスを付けて 5×5 のキーとタブを隠す。
- 画面の高さが、モードを入れてからの最小値の 1.3 倍を超えたら（端末のキーボードが閉じた）画面のキーに戻す。幅が変わったとき（回転）は基準を取り直す。高さが縮んだときは履歴を最新までスクロールする。

プレビュー（exprChanged、runPreview）

- 変更のたびに入力中の式を 0.5 秒まとめて保存し、入力欄の高さを合わせる（最大 110 px × 文字の大きさの倍率）。

- 空欄またはプレビューがオフなら表示を消す。そうでなければ 160 ms 待って計算する（timeout 1500、max_chars 450000、tail_chars 20000。版 1.3.0 で確定と同じ長さにした。それまでは max_chars 6000）。
- 計算中のプレビューは同時に 1 件だけ。その間の変更は終わってから 1 回だけ計算し直す。結果は、入力欄がそのままで確定中でないときだけ表示する。
- エンジンが実行を見送った式（deferred）には「＝ を押すと実行します（定義など）」と表示する。

文字の大きさ（applyTextSize）

設定 text_size（normal・large・xlarge）を html 要素の data-text-size 属性に書き、CSS の変数 --fs（1・1.18・1.38）で、履歴（式・結果・メッセージ）、解釈した式、入力欄、プレビュー、メッセージの文字の大きさを変えます。キーとパネルの文字は変えません。入力欄の高さの上限（110 px）も同じ倍率で広げます。起動時（boot）と設定を変えたとき（settingsChanged）に適用します。

3.9 履歴と結果の再利用

確定の流れ（commit）

1. 入力欄が空、または確定中なら何もしない。エンジンの準備（init）が終わっていなければ「計算エンジンを準備中です」と表示する。
2. 式を normalizeJa で変換し、予定していたプレビューを取り消して「計算中…」と表示する。1.2 秒たっても終わらなければ「計算中…（ここをタップで中止）」に変える。タップすると QalcNative.cancel() を呼び、「中止しています…」と表示する。
3. calc を store=true、timeout 20000、max_chars 450000（MAX_RESULT。実部と虚部が 20 万桁の複素数、3 桁区切り付きの 20 万桁が入る長さ）、tail_chars 20000（TAIL_RESULT）で送る。
4. 失敗（ok=false）なら「計算を中止しました」（code=cancelled）または error の文を、timeout なら「時間がかかりすぎたため計算を中止しました」を、結果が空なら最初のエラーメッセージ（日本語化）か「式を確認してください」を赤で表示する。履歴には残さない。
5. 成功したら履歴の項目を作って末尾に追加し（最大 300 件、古いものから消す）、300 ms まとめて保存する。
6. 入力欄が送ったときのままなら空にする。計算中に何か入力されていたら消さずに残し、プレビューをやり直す。
7. 履歴を描き直して最新までスクロールし、画面読み上げ用の #status に「式、結果」を入れる（読み上げるのは新しい結果だけ）。

履歴の項目（qalc.history の 1 件）

項目	型	内容
expr	文字列	送った式（変換後）
result	文字列	画面に出す結果の文字列。全部の桁を持つ（450,000 バイトを超えてエンジンが切ったものは、先頭の 430,000 バイトと「…」。末尾は tail）。fitHistory() が新しい項目から長さを足していき、合計が 1,000,000 文字（HISTORY_BUDGET）に達したあとの項目は 2,000 文字（SHORT_RESULT）で切って「…」を付ける（版 1.2.0。それまでは各 20,000 文字）
approx	真偽	近似値なら true
cmp	真偽	比較・方程式の結果（x = 2、true など）なら true
base	数値	計算したときの表示の基数
msgs	配列	警告とエラーのメッセージ（info を除き最大 3 件。各 {type, text}。text はエンジンの英語の文で、日本語の画面では表示するときに日本語にする）。エンジンが切った結果には、警告 The middle of this long result is not shown (N digits) を 1 件加える（日本語の画面では「長い結果のため、途中を省略しています（全 N 桁）」）
t	数値	確定した時刻（ミリ秒）
input	文字列（任意）	挿入用の正確な値。ans イベントの input（20,000 文字（MAX_INPUT）以下のとき。容量を超えた古い項目では 2,000 文字を超える input を捨てて big にする）
cut	真偽（任意）	result を切った印（履歴の容量で切ったとき、またはエンジンが切ったとき）
tail	文字列（任意）	エンジンが切った長い結果の末尾（20,000 バイトまで。版 1.3.0）。画面は「先頭 … 末尾」と表示する。容量を超えて result を短くした項目では消す
big	真偽（任意）	input が長すぎて保存しなかった印

ans イベントとの対応

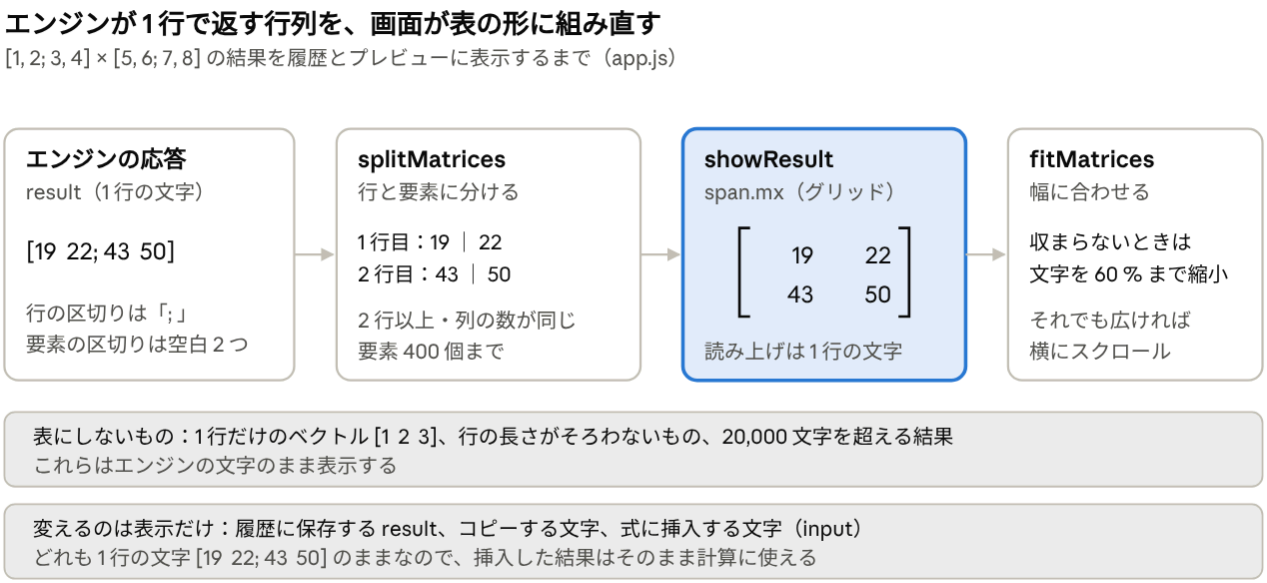
エンジンは結果の応答の直後に、同じ id を付けた ans イベントを出します。画面は今回の起動で作った項目を id で記憶し（sessionEntries）、イベントが届いたらその項目に input を付けて保存し直します。イベントが応答より先に届いたときは、最大 20 件まで預かっておき

（earlyInputs）、項目を作るときに使います。

表示の変換（displayText、displayResult）

- 桁区切りの設定のとき、エンジンは数字を狭い空白で区切る（20 文字までの数は U+202F、それより長い数は U+2009。小数部も区切り、数と単位の間にも U+202F を入れる）。画面は整数部の区切りだけを「,」にし（1,234,567）、小数部の区切りと単位の前の空白はそのままにする（3.141 592 654、12.345 km。版 1.3.0 で、小数部にカンマを入れないように直した）。
- エンジンが切った長い結果（tail があるもの）は、先頭の終わりの「…」を除いて「先頭 … 末尾」と表示する（shownText）。末尾が小数部の途中から始まるときは、末尾の区切りはそのままにする。
- 日付だけの結果（"2026-11-09" など）は引用符を外し、T を空白にする。
- 比較・方程式の結果はそのまま、近似値は「≈ 」（結果に ≈ が含まれるときは付けない）、それ以外は「= 」を前に付ける。

行列の表表示（splitMatrices、showResult、fitMatrices）



行列の表表示 · エンジンの 1 行の文字から表の形まで

- エンジンは行列を 1 行の文字で返す（[1 2; 3 4]。行の区切りは「; 」、同じ行の要素の区切りは空白 2 つ。数や名前 1 つでない要素は括弧に入る）。履歴の結果とプレビューでは、2 行以上の行列を表の形に描く。
- splitMatrices は、結果の文字を先頭から調べ、「[」から対応する「]」までを、括弧の深さと " " で囲まれた文字列を見ながら行と要素に分ける。2 行以上で、どの行も要素の数が同じで、空の要素がなく、要素が全部で 400 個以下のものだけを行列とみなす。1 行だけのベクトル（[1 2 3]）や、行の長さがそろわないものは文字のまま表示する。
- 表示を遅くしないため、結果が 20,000 文字を超えるとき、「;」を含まないとき、「[」を 200 個調べたあとは、それ以上調べない。

- showResult は、行列の部分を span.mx（CSS のグリッド。列の数を grid-template-columns で指定）にして、要素を 1 つずつ入れる。前後の文字（「= 」、「x = 」、行列の間の演算子など）はそのまま並べる。画面読み上げ用に role="img" と、元の 1 行の文字を aria-label に付ける。
- fitMatrices は、表示する場所より幅の広い行列の文字を小さくする（60 % まで。最大 4 回計算し直す）。それでも収まらない行列は横にスクロールできる。
- 変えるのは表示だけで、履歴に保存する result、コピーする文字、式に挿入する文字（input）は 1 行の文字のまま。

長い結果（版 1.2.0。fitResultBox、fitHistory、clip）

- 精度の上限は 200,000 桁（MAX_PRECISION。画面の app.js とエンジンの qalc-engine.cc に同じ値）。QCalc Pro が使えないときは 1,000 桁（FREE_MAX_PRECISION。3.20）。設定の欄に範囲外の数を入れると 2〜200,000 に収め、エンジンも同じ範囲に収める。
- 履歴の結果（.h-res）は、高さが --res-max を超えると枠の中でスクロールする。fitResultBox が履歴の欄の高さの 70 %（最小 96 px）を --res-max に入れ、ResizeObserver（ないときは resize）で計り直す。履歴そのものは、枠の外を動かしてスクロールできる。
- 長押しのメニューの見出しは、式と結果をそれぞれ先頭 300 文字に、読み上げ用の文（#status）はそれぞれ先頭 200 文字にする（clip。続きは「…」）。メニューの「結果をコピー」は全部の桁をコピーする。
- fitHistory は、確定のあと、履歴を変えたあと、起動して履歴を読んだあとに呼ぶ。新しい項目から result・tail・input・expr の長さを足し、合計が 1,000,000 文字に達したあとの項目は result を 2,000 文字で切り（cut）、tail を消す。一番新しい結果は切らない。保存する JSON は、もっとも大きいときでも約 210 万文字（全部の桁で残す分が最大約 150 万、切った 300 件分が約 60 万。StateStore が書く上限 400 万文字より小さい）。
- エンジンが切った結果（truncated）は cut にし、式に挿入しない。sessionEntries（ans イベントを待つ項目）はイベントが届いたら消し、20 件までにする（長い結果をメモリに残し続けないため）。
- 版 1.3.0：プレビューも確定と同じ max_chars 450,000、tail_chars 20,000 で計算し、全部の桁を表示する（それまでのプレビューは 6,000 バイトまでで、確定した結果と見た目が違っていた）。450,000 バイトを超える結果（100000! の 456,574 桁など）は、プレビューと履歴のどちらも、先頭の 430,000 バイト、「…」、末尾の 20,000 バイトを表示し、メッセージ「長い結果のため、途中を省略しています（全 N 桁）」を添える（数字が文字の 6 割以上なら桁数、そうでなければ文字数。cutMessage）。

履歴の操作

操作	動作
式の行をタップ	その式で入力欄を置き換え、カーソルを末尾に置く（入力中だった式は「元に戻す」で戻せる。3.8）
結果の行をタップ	結果をカーソル位置に挿入（下の規則）
長押し（480 ms、10 px 以上動くと取り消し）	「結果をコピー」「式をコピー」「式を編集」「結果を式に挿入」「この行を削除」のメニュー。コピーする結果は画面と同じ表示形式（行列は1行の文字）。削除した行は、通知「1行を削除しました」の「元に戻す」（5秒間）で元の位置に戻せる
メニュー「履歴を消去」	確認のあとすべて消す（エンジンの ans は残る）。通知「履歴を消去しました」の「元に戻す」（6秒間）で戻せる。その間に確定した計算は、戻した履歴の後ろに残る。履歴が空のときは「履歴はありません」と表示するだけ

履歴の表示位置：履歴を最新（下端から 24 px 以内）までスクロールしているあいだは、入力欄やプレビューが伸びて履歴の高さが変わっても、最新の行を表示し続けます（ResizeObserver）。上にスクロールして古い履歴を読んでいるときは動かしません。

最初の画面（履歴が空のとき）

renderHistory は、履歴が1件もないとき、履歴の欄に、使い方の説明（「式を入力して = を押すと、ここに結果が残ります」など）、式の例 4 つ（STARTERS：60 mph to km/h、100 USD to EUR（日本語の画面では 100 USD to JPY）、solve(x^2 − 5x + 6 = 0)、diff(x^3)）、「式の例・使い方を見る」ボタンを表示します。例をタップすると入力欄に入り（setExpr）、プレビューに結果が出ます。この画面は上から読むので、スクロール位置を先頭にします（結果があるときは最新の行が見える下端に合わせます）。

挿入する文字列の規則（insertText）

1. cut または big の項目は挿入しない。「この結果はそのまま挿入できません（ans キーで使えます）」と表示する。
2. input があればそれを使う。例：画面が「2 + 1/3 ≈ 2.333333333」なら 7/3、16 進表示の「0xFF」なら 255、「x = 2 or x = −2」なら [2 −2]。
3. 前の版で保存した履歴（input がない）は表示の文字列から作る。基数が 10 以外なら 0x・0b・0o を付ける（12 進とローマ数字は挿入しない）。「名前 = 値」の比較結果は値だけを使い、or・and を含むものは挿入しない。それ以外は「≈」「=」より前の部分を使う。

4. 数1つ（符号、小数点、E の指数を含む）、名前1つ、[ ] の配列、" " の文字列はそのまま、それ以外は ( ) で囲んで挿入する（前後の演算子と結びつかないように）。

3.10 グラフ機能の詳細（2D・3D）

2D・3D のどちらも、値の計算はエンジン（libqalculate）、描画と操作は画面（canvas）が担当します。gnuplot や WebGL は使いません。

2D グラフ（y = f(x)）

2D の点は libqalculate の expressionToPlotVector で計算します。

画面から送る要求

項目	値
exprs	入力欄の1行を1つの式として最大6つ（空行は除く。各行に normalizeJa をかける）
min / max	x の範囲の入力欄の文字列（初期値 −10 と 10。pi などの式も可）
steps	600
var	x
timeout	12000（すべての式で共通の持ち時間）
opts	計算画面と同じ設定

エンジンの処理（cmd_plot）

1. exprs がなければ「exprs missing」、12 個を超えれば「too many expressions」を返す。steps は 10〜4000（省略時 300）、timeout は 100〜120000 ms（省略時 8000）に収める。
2. 近似計算を強制し、精度が 20 桁を超える設定でも 20 桁で計算する（点は double で送るため）。終わったら精度を戻す。
3. min と max をそれぞれ 2 秒の制限つきで計算し、数にできない、入れ子が深すぎる、min ≥ max のときは「invalid range」を返す。
4. 式ごとに、残り時間（timeout から経過時間を引いたもの）が 50 ms 以上あれば点を計算する。値にできない y は null、x にできない点は飛ばす。時間切れや文字列でない式は空の系列になる。
5. series（式ごとの {x:[…], y:[…]}）と messages を返す。

表示範囲と目盛り

- 最初の表示範囲（autoView）：x は点の最小〜最大。y は点が 50 個以上あれば下から 2 %〜98 % の位置の値（tan(x) のような発散で全体が潰れないように）、少なければ最小〜最大。幅がほぼ 0 なら少し広げ、上下に 8 % の余白を付ける。点が 1 つもなければ −10〜10。
- 目盛りの間隔（niceStep）は 1・2・5 × 10^n から選び、横は約 90 px、縦は約 70 px に 1 本（最低 3 本）。目盛りの文字は隣と区別できる桁数で書き、とても大きい・小さい値は E 表記にする。
- 安全対策：線の本数が 400 を超える、または位置が 2^{52} を超えるときは目盛りを描かない（gridValues）。拡大・縮小・移動のあとの範囲は、幅が値の大きさの 10^{-12} 倍より広く 10^{300} より狭いときだけ採用し（viewOk）、そうでなければ「これ以上は拡大・縮小できません」と表示する。

線の描画

- 6 色を順に使い、太さ 2.2 px。端末の画素密度（devicePixelRatio）に合わせて描く。
- y が null・無限大の点で線を切る。隣の点との差が画面の高さの 1.5 倍を超えるときも切る（漸近線に縦線を引かないため）。
- 凡例には式を表示し、値が 1 つもない式には「(値なし)」を付ける。

操作

操作	動作
「描画」	式と範囲を保存し、入力した範囲で計算し直す。前のグラフでタップした点の印と座標の表示は消す
1 本指でドラッグ	表示範囲を動かす
2 本指でつまむ・広げる	2 本の指の中点を中心に拡大・縮小する
＋ ／ −	中心を基準に 2 倍拡大 ／ 2 倍縮小
↺	入力した範囲で計算し直し、表示範囲を初期に戻す
タップ	いちばん近い点に印を付け、「式：x = …, y = …」（有効数字 10 桁）を表示する。近くに点がなければ消す

範囲外の再計算（refetchIfNeeded）

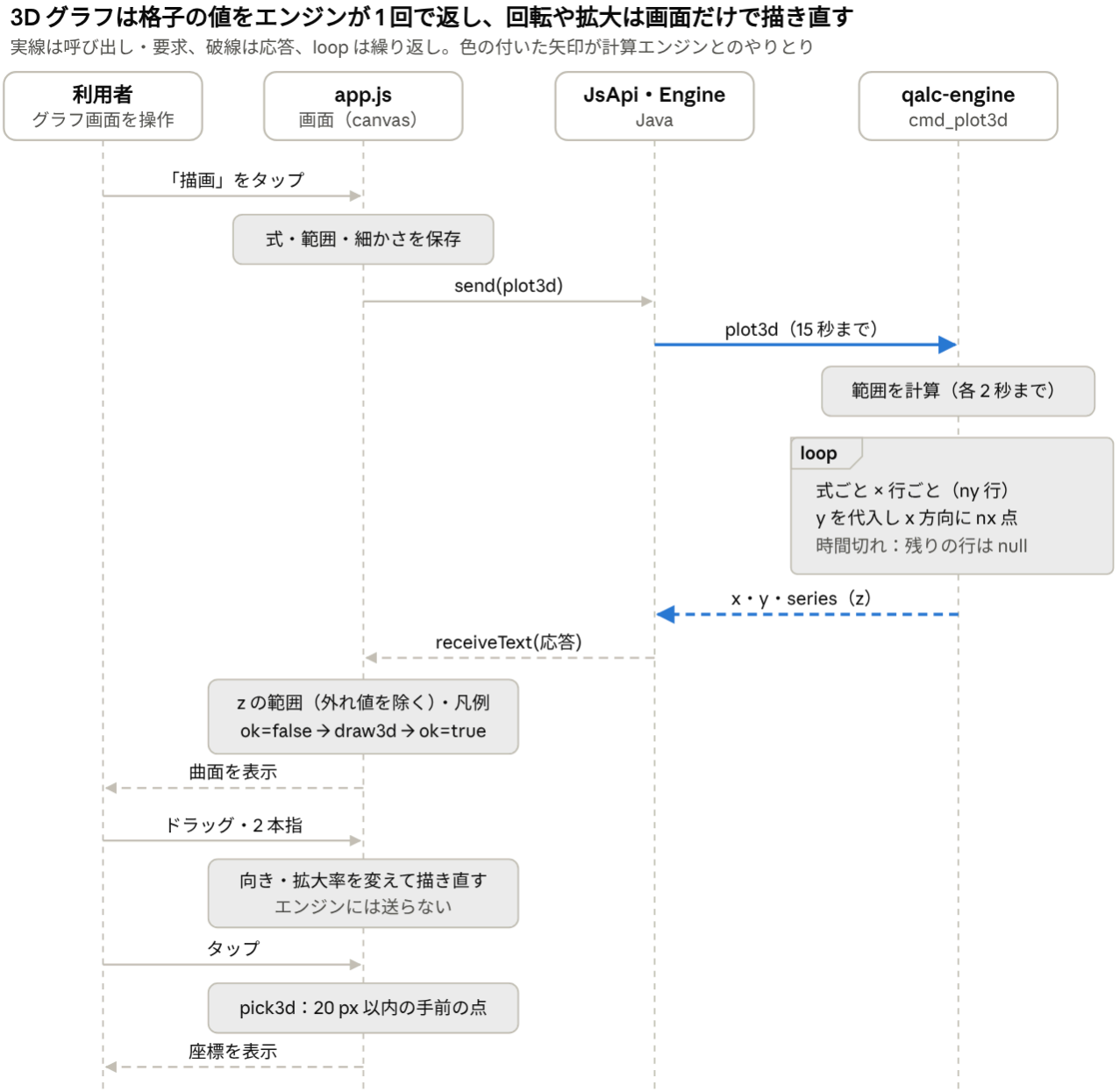
移動・拡大・縮小の 0.35 秒後、表示範囲が計算済みの x の範囲から幅の 5 % 以上はみ出したとき、または幅が計算済みの 1/6 より狭くなったときは、表示範囲の左右に幅の半分ずつを加えた範囲で計算し直します（表示範囲はそのまま）。範囲の数は「e」を「E」にして送ります（小文字の e はネイピア数と読まれることがあるため）。

前回の内容と安全な再開

式と範囲は qalc.plot.exprs・min・max に保存し、次に開いたときに自動で描きます。描画の前に qalc.plot.ok を false、描けたら true にするので、前回の描画中にアプリが止まったときは自動では描かず「「描画」を押すとグラフを描きます」と表示します。エラー「invalid range」は「x の範囲を確認してください」と表示します。

3D グラフ（$z = f(x, y)$）

3D グラフは、エンジンが格子点の z をまとめて計算し（plot3d）、画面が canvas に曲面を描きます。回転・拡大・座標の表示は画面の中だけで行い、エンジンには送りません。WebGL は使わず、奥の四角形から順に塗る方法（画家のアルゴリズム）で描きます。



シーケンス図·3D グラフの計算と描画

画面から送る要求（plot3d）

項目	値
exprs	入力欄の1行を1つの式として最大4つ（空行は除く。各行に normalizeJa をかける）。変数は x と y
xmin・xmax・ymin・ymax	範囲の入力欄の文字列（初期値 −5 と 5。pi などの式も可）
nx・ny	「細かさ」の値：25（粗い）・41（標準、初期値）・61（細かい）。x と y で同じ
timeout	15000（すべての式で共通の持ち時間）
opts	計算画面と同じ設定

エンジンの処理（cmd_plot3d）

1. exprs がないか空なら「exprs missing」、4 個を超えれば「too many expressions」を返す。nx・ny は 2〜121（省略時 41）、timeout は 100〜120,000 ms（省略時 15,000）に収める。
2. 近似計算を強制し、区間演算を止め、複素数を許す。精度が 20 桁を超える設定でも 20 桁で計算し（値は double で送るため）、終わったら戻す。
3. xmin・xmax・ymin・ymax をそれぞれ 2 秒の制限つきで計算し、数にできない、入れ子が深すぎる、最小 ≧ 最大のときは「invalid x range」または「invalid y range」を返す。
4. x は xmin〜xmax を nx 点に等分する（最後の点は xmax ちょうど）。y は (ymax − ymin) ÷ (ny − 1) を libqalculate の数のまま積み上げて ny 個の値を作る。
5. 式ごとに、残り時間が 50 ms 以上あれば startControl(残り時間) で制限をかけてから式を解釈する（書き間違いのメッセージは返す）。個々の点のメッセージは止め、行ごとに y を代入して generateVector で x 方向の nx 点を計算する。
6. 実数だけを値にする（虚部が実部の 10^{-12} 倍以下なら実数とみなす）。複素数・未定義・無限大・単位つきの値は null。時間切れになったら残りの行を null で埋め、complete=false にする。
7. x（nx 個）、y（ny 個）、series（式ごとの z と complete）、messages を返す。数は 12 桁で書く。

描画（draw3d）

- 座標：x・y を −1〜1、z を −0.72〜0.72 の箱に合わせる。z の範囲（zRange3d）は実数の値の最小〜最大。ただし値が 50 個以上あり、全体の幅が 1 % 点〜99 % 点の幅の 3 倍を超えるときは、1 % 点〜99 % 点に幅の 15 % の余白を付けた範囲にする（1/(x y) のような極で曲面が平らに潰れないように）。範囲の外に出る格子点を含む四角形は描かない。

- 投影（正射影）：方位角 az、仰角 el、拡大率 zoom、箱の座標 (u, v, w) から、xr = u·cos az − v·sin az、yr = u·sin az + v·cos az、画面の x = cx + xr·S、y = cy − (w·cos el + yr·sin el)·S、奥行き = yr·cos el − w·sin el。S = min(W ÷ 3.3, H ÷ (2·(0.72·|cos el| + 1.42·|sin el|) + 0.75)) × zoom。初期値は az = −0.62、el = 0.52、zoom = 1。
- 箱：奥にある 3 面を薄く塗り、目盛り線（niceStep で約 4 等分、最大 40 本）を引く。軸は手前の床の 2 辺と左端の縦の辺に引き、目盛りの数値と x・y・z を書く（床の高さの z の目盛りは x・y の目盛りと重なるので書かない）。
- 面：格子の四角形ごとに 4 頂点の奥行きの和を求め、奥から順に塗る。陰影は、2 本の対角線の外積で求めた法線と光の向き (−0.42, 0.62, −0.66)（画面の左上・手前から）の内積の絶対値。
- 色：式が 1 つなら高さに応じた viridis の色（明るさ 0.5 + 0.5 × 陰影）、2 つ以上なら式ごとの色（明るさ 0.42 + 0.58 × 陰影）。格子が 45 × 45 以下なら網目の線も引く。凡例は、式が 1 つなら色の帯と z の最小・最大、2 つ以上なら式ごとの色。値が 1 つもない式には「(値なし)」を付ける。
- 速さ：格子点の投影は式ごとに型付き配列へ 1 回だけ計算し、配列は描き直しのたびに使い回す。ドラッグ中の描き直しは requestAnimationFrame で 1 フレームに 1 回にまとめる。

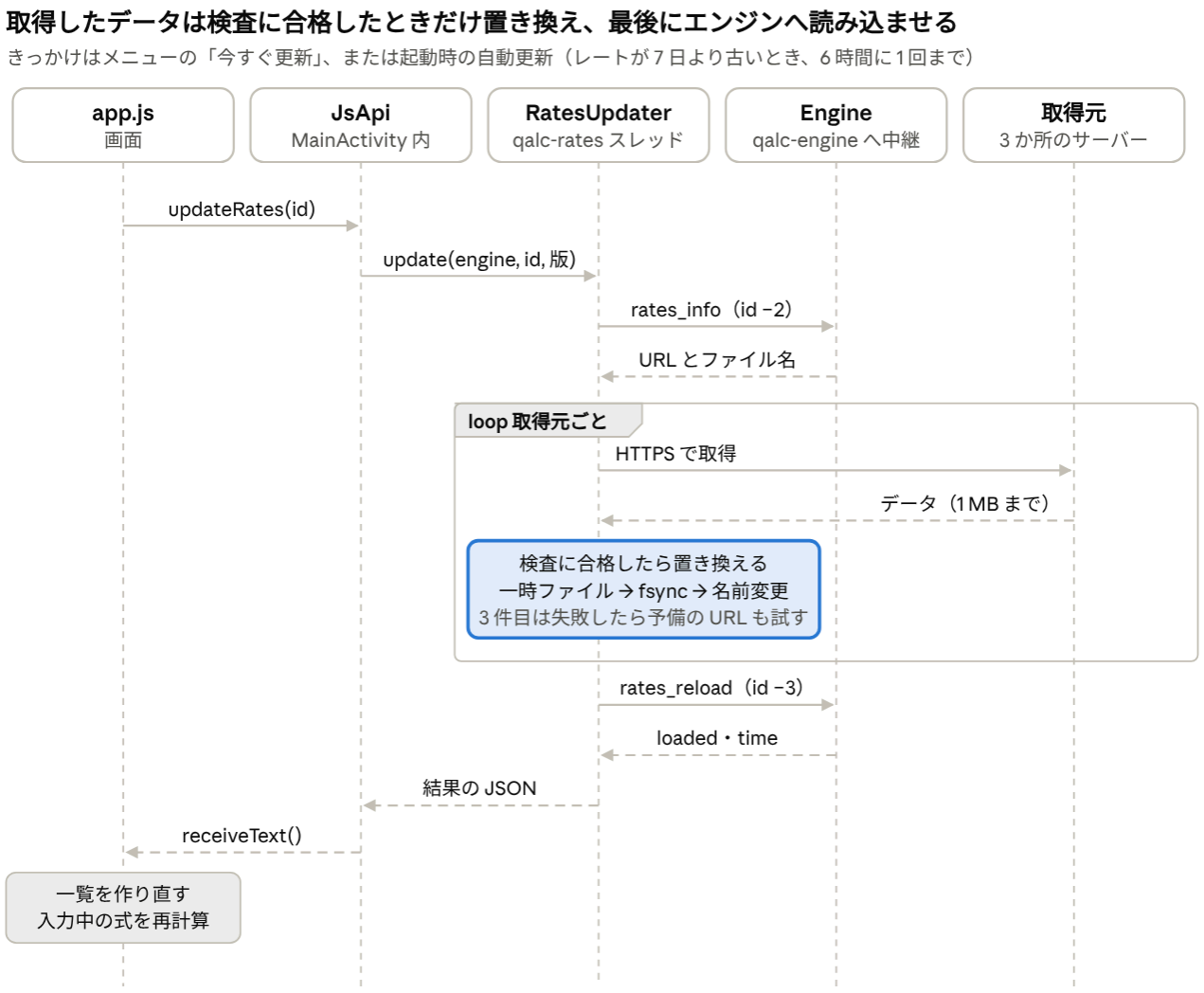
操作（3D）

操作	動作
「描画」	式・範囲・細かさを保存し、計算し直す
1 本指でドラッグ	回転。横 1 px で方位角、縦 1 px で仰角が 0.01 rad 変わる（仰角は ±1.5 rad まで）
2 本指でつまむ・広げる	拡大・縮小（0.4〜6 倍）
＋ ／ −	1.25 倍 ／ 0.8 倍
↺	向きと大きさを初期に戻す
タップ	指から 20 px 以内の格子点のうち手前のものに印を付け、「式：x = …, y = …, z = …」（有効数字 10 桁）を表示する。近くになければ消す

2D と 3D の切り替え：グラフ画面の上のボタンで切り替え、最後に表示した種類を qalc.plot.mode に保存して次回もその種類で開きます。入力と図はパネルを閉じても残り、2D は初めて表示したときに計算します。3D の式・範囲・細かさは qalc.plot3d.* に保存し、2D と同じく描画の前後に完了印 qalc.plot3d.ok を書き換えるので、前回の描画中にアプリが止まったときは自動では描きません。画面の大きさが変わったときは 2D・3D の両方を描き直します。

3D グラフは QCalc Pro の機能です（版 1.3.0。3.20）。Pro が使えないときは 3D のボタンに PRO の印を付け、押すと Pro の案内を出して 2D のままにします。保存された種類が 3d でも 2D で開きます（setMode）。

3.11 為替レート更新の詳細



シーケンス図 · 為替レートの更新

公式の Qalculate! と同じ 3 か所の取得元から、libqalculate が使うファイル名のまま取得します。エンジンは通信機能なし（libcurl なし）でビルドしているため、ダウンロードは Java（RatesUpdater）が行い、エンジンには取得後に読み込みだけを指示します。URL と保存先は libqalculate が返すものを使います。

取得元と検査の条件

番号	取得元	保存するファイル	合格の条件
1	欧州中央銀行（https://www.ecb.europa.eu/stats/eurofxref/eurofxref-daily.xml）	eurofxref-daily.xml	`<Cube` と `currency=` を含み、`</gesmes:Envelope>` で終わる（途中で切れていない）
2	Coinbase（https://api.coinbase.com/v2/prices/spot?currency=EUR）	btc.json	`"amount":` と `"currency":` を含み、JSON のオブジェクトとして読める
3	fawazahmed0 currency-api（https://cdn.jsdelivr.net/npm/@fawazahmed0/currency-api@latest/v1/currencies/eur.json。失敗したら https://latest.currency-api.pages.dev/v1/currencies/eur.json）	rates.json	`: {` と `"eur": 1,`（または空白なし・大文字の同じ書き方）を含み、JSON のオブジェクトとして読める

検査は libqalculate 自身のダウンロード処理と同じ条件に、「最後まで届いたか」の確認を加えたものです。途中で切れたデータで置き換えると、libqalculate は黙って組み込みの古いレートに戻ってしまうためです。

手順（RatesUpdater.update）

1. rates_info（id −2）で、エンジンから 3 つの取得元の番号、URL、保存先を受け取る。
2. 取得元ごとに HTTPS でダウンロードする。接続 15 秒、読み込み 20 秒、1,000,000 バイトまで。リダイレクトに従い、User-Agent は QCalc/1.3.0 (Android; libqalculate 5.13.2)（1.3.0 の部分はアプリの versionName。英数字・「.」・「-」だけを 32 文字まで使い、取得できないときは 0。テスト版は 1.3.0-debug）。HTTP 200 以外は失敗。
3. 検査に合格したら、一時ファイル（.tmp）に書いて fsync し、名前を変えて置き換える。不合格・失敗のときは元のファイルを残し、取得元のホスト名を failed に入れる。
4. rates_reload（id −3）でエンジンに読み込み直させる。1 件以上更新したのに loaded=false なら失敗にする。
5. 結果（ok、updated、total、failed、time、失敗時は error）を画面に返す。画面は最終更新の日時と「2 / 3 件の取得元から更新しました。」のような結果を表示し、成功したら単位・通貨などの一覧を作り直して、入力中の式を計算し直す。

同時実行の防止：update は synchronized なので、自動と手動の更新が重なっても同じファイルを同時に書きません。画面では更新中は「今すぐ更新」ボタンを押せなくします。

自動更新：起動時に、設定「為替レートを自動更新」がオンで、init の rates_time が 7 日より前（または不明）で、前回の試行から 6 時間以上たっていれば実行します。成功したら「為替レートを最新に更新しました」と表示します。失敗しても何も表示しません。

3.12 保存データの形式

アプリのデータはすべて端末内のアプリ専用領域（/data/user/0/io.github.hiroshitsu.qcalc/。テスト版は io.github.hiroshitsu.qcalc.debug の別の領域）にあり、ほかのアプリからは読めません。インターネットに送るデータはありません（為替レートの取得はダウンロードだけです。QCalc Pro の購入と確認は、端末の Google Play ストアのアプリを通して行います。3.20）。

場所（アプリ専用領域からの相対）	書く部品	形式	内容
files/state/qalc.history.json	画面 → StateStore	JSON の配列	計算履歴（最新 300 件。項目は 3.9）
files/state/qalc.settings.json	画面 → StateStore	JSON のオブジェクト	設定 19 項目（版 1.1.0 で text_size、版 1.3.0 で lang と theme を追加。ファイルにない項目は既定値を使う。lang は使っている言語の控えで、言語は qalc.lang で決まる）
files/state/qalc.lang.json	画面 → StateStore	JSON の文字列（"en"・"ja"）	利用者が「Language / 言語」で選んだ表示の言語（版 1.3.0。3.21）。ないときは英語
files/state/qalc.draft.json	画面 → StateStore	JSON の文字列	入力中の式
files/state/qalc.plot.mode.json	画面 → StateStore	JSON の文字列（"2d"・"3d"）	最後に表示したグラフの種類
files/state/qalc.plot.exprs.json、qalc.plot.min.json、qalc.plot.max.json	画面 → StateStore	JSON の文字列	2D グラフの式（改行区切り）と x の範囲
files/state/qalc.plot3d.exprs.json、qalc.plot3d.xmin.json・xmax.json・ymin.json・ymax.json、qalc.plot3d.res.json	画面 → StateStore	JSON の文字列	3D グラフの式（改行区切り）、x・y の範囲、細かさ（"25"・"41"・"61"）
files/state/qalc.plot.ok.json、qalc.plot3d.ok.json	画面 → StateStore	JSON の真偽	前回の 2D・3D グラフの描画が最後まで終わったか
files/state/qalc.rates.try.json	画面 → StateStore	JSON の数値	為替レートの自動更新を最後に試した時刻（秒）

場所（アプリ専用領域からの相対）	書く部品	形式	内容
WebView の localStorage	画面	同じキーと内容	上の各項目の控え（以前の版と PC での確認用）
shared_prefs/engine.xml（キー ans）	Engine	文字列	最後の ans の値（エンジンを起動し直したときに戻す）
shared_prefs/pro.xml	Pro	SharedPreferences	QCalc Pro の状態（版 1.3.0。3.20）：owned（購入済み）、price（価格の文字）、pre_pro（課金のない版で起動した印）、era（課金のある版を起動した印）、legacy（Pro が始まる前からの利用者）、test_locked（テスト版で Pro を外して見る）
files/qalculate/definitions/variables.xml、functions.xml、units.xml、datasets.xml	エンジン（libqalculate）	libqalculate の定義 XML	ユーザー定義の変数・関数
files/qalculate/broken-definitions/〈ファイル名〉.〈時刻〉	エンジン	元のまま	起動時に読めなかった定義ファイルの退避先
files/qalculate/eurofxref-daily.xml、btc.json、rates.json	RatesUpdater	取得元の形式のまま	為替レート（欧州中央銀行、Coinbase、fawazahmed0 currency-api）
利用者が選んだ場所（システムのファイル選択画面）	MainActivity	CSV（UTF-8、BOM 付き）	書き出した履歴（版 1.3.0。3.23）。アプリ専用領域の外にあり、アプリを消しても残る

例：設定（初期値）と履歴 1 件

```
{"angle":"rad","approx":"auto","fraction":"auto","precision":10,"max_decimals":-1,"base":10,"notation":"auto","units":"optimal","mixed_units":true,"complex":"rect","grouping":false,"structuring":"simplify","preview":true,"haptics":true,"auto_ans":true,"auto_rates":true,"text_size":"normal","lang":"en","theme":"blue"}
[{"expr":"7/3","result":"2 + 1/3 ≈ 2.333333333","approx":false,"cmp":false,"base":10,"msgs":[],"t":1794216542706,"input":"7/3"}]
```

書き込みの規則

- StateStore（画面の状態）：名前は英小文字・数字・「.」「_」「-」の 1〜64 文字（画面のキーは小文字にし、それ以外の文字を「_」に置き換えて渡す）。1 件 4,194,304 文字まで、8 MB を超えるファイルは読まない。書き込みは専用の 1 本のスレッドで行い、まだ書いていない同じ名前の内容は新しいもので置き換える（読み出しは書き込み待ちの内容を優先）。一時ファイル（.tmp）に書いて fsync してから名前を変えるので、途中で止まっても前の内容が残る。
- 画面の読み出しは StateStore を先に見て、なければ localStorage を見る（localStorage だけに保存していた以前の版からの引き継ぎ）。書くときは両方に書く。
- ans：値が空、または 500,000 文字を超えるときは保存しない（null にする）。SharedPreferences の apply() で非同期に書く。
- ユーザー定義：変更のたびに 4 つのファイルを書き直す。一時ファイルは definitions/ の外（files/qalculate/.〈ファイル名〉.tmp）に作る（definitions/ の中のファイルは起動時にすべて読み込まれるため）。fsync してから名前を変える。起動時に XML として読めないファイルは broken-definitions/ に移し、init の defs_error で画面に知らせる。
- 為替レート：取得したデータを検査（3.11）してから、一時ファイル + fsync + 名前の変更で置き換える。

消えるとき

操作	消えるもの
メニュー「履歴を消去」	履歴だけ（ans、設定、定義は残る）
ユーザー定義の画面で「削除」	その変数・関数だけ
Android の設定でアプリのデータを削除、またはアンインストール	すべて
上書き更新（Google Play からの更新。テスト版は同じテスト用の鍵で署名した新しい APK）	何も消えない

allowBackup=true なので、端末のバックアップ設定によってはこれらのデータが Android のバックアップに含まれます。QCalc Pro の購入は Google アカウントに記録されるので、アプリのデータを消しても入れ直しても、同じアカウントなら起動時の問い合わせで使えるようになります。ただし「以前からの利用者」の印（legacy）は端末の中の記録なので、アプリのデータを消すと戻りません（3.20）。

3.13 時間制限と上限値の一覧

時間制限は 3 段構えです。 いちばん内側がエンジン自身の時間制限、次がエンジンの監視スレッド、外側が Java の待ち時間と画面の最後の安全策です。内側が働けば外側は使われません。

要求	エンジンの制限（timeout）	監視スレッド（timeout + 3 秒）	Java の待ち時間（timeout × 式の数 + 10 秒）	画面の最後の安全策
プレビュー	1.5 秒	4.5 秒	11.5 秒	120 秒
確定（=）	20 秒	23 秒	30 秒	120 秒
グラフ（式 6 つ）	12 秒（全部の式で共通）	15 秒	82 秒	204 秒
3D グラフ（式 4 つ）	15 秒（全部の式で共通）	18 秒	70 秒	180 秒
一覧・定義・init など（timeout なし）	—	—	30 秒	120 秒
ans の復元（set_ans、id −1）	15 秒	—	20 秒	—
為替レートの更新	—	—	rates_info・rates_reload に各 30 秒	300 秒

画面の最後の安全策は max(120 秒, 2 × timeout × 式の数 + 60 秒) です（timeout がない要求は 30 秒として計算）。

エンジン（qalc-engine.cc）

項目	値
calc の timeout	100〜120,000 ms（省略時 3,000）
calc の max_chars	100〜10,000,000 バイト（省略時 200,000）。超えた分は文字の途中で切らずに捨て「…」を付ける。tail_chars は 0〜1,000,000 バイト（省略時 0）で、2 倍が max_chars 以下のときだけ末尾を返す（版 1.3.0）
parsed（解釈した式）の長さ	2,000 バイト

項目	値
plot の timeout / steps / 式の数	100〜120,000 ms（省略時 8,000）/ 10〜4,000（省略時 300）/ 12 個まで
plot の範囲の計算	min・max それぞれ 2 秒。式 1 つの計算を始めるには残り 50 ms 以上が必要
plot3d の timeout / 格子 / 式の数	100〜120,000 ms（省略時 15,000）/ nx・ny それぞれ 2〜121（省略時 41）/ 4 個まで
plot3d の範囲の計算	xmin・xmax・ymin・ymax それぞれ 2 秒。式 1 つの計算を始めるには残り 50 ms 以上が必要
plot3d の値	実数だけ（虚部が実部の 10^{-12} 倍以下なら実数とみなす）。12 桁で送る
plot・plot3d の精度	20 桁まで（設定がそれより大きくても）
入れ子の深さ	かっこ（([{）300 段まで、^ は 1,000 個まで
監視スレッドの確認間隔	0.2 秒
終了コード	0：入力が閉じた、3：監視スレッドが時間切れで応答した、4：強制中止のあとの再起動、5：処理できない例外（メモリ不足など）

Java（Engine・StateStore・RatesUpdater・MainActivity）

項目	値
保存する ans の長さ	500,000 文字まで（200000 桁の近似値が約 40 万文字）
StateStore の 1 件	4,194,304 文字まで（読むファイルは 8 MB まで）
ダウンロード	接続 15 秒、読み込み 20 秒、1 ファイル 1,000,000 バイトまで
戻る操作の応答待ち	0.8 秒（応答がなければアプリを背面へ回す）
画面表示の再起動	15 秒以内に 2 回止まったら再起動せずに案内を表示
保存するファイル（saveFile）	名前は英数字で始まる英数字・「.」「_」「-」の 100 文字まで、種類は text/csv、内容は 16,777,216 文字まで
QCalc Pro の問い合わせ	起動時と、画面に戻るたび（onResume）。Pro の画面を開いたときも

画面（app.js）

項目	値
プレビューの開始待ち	160 ms（max_chars 450,000、tail_chars 20,000）
確定の max_chars・tail_chars	450,000・20,000
「ここをタップで中止」の表示	1.2 秒後
履歴	300 件。結果は新しいものから合計 1,000,000 文字までは全部の桁、それより古い長い結果は 2,000 文字まで。メッセージは 3 件まで
保存のまとめ時間	履歴 300 ms、入力中の式 500 ms
先に届いた ans イベントの預かり	20 件
キーの繰り返し	420 ms 後から 70 ms ごと
長押し	480 ms（10 px 以上動くと取り消し）
振動	8 ms
有効桁数の設定	2〜200,000（QCalc Pro が使えないときは 1,000 まで）
為替レートの自動更新	7 日より古いとき。試すのは 6 時間に 1 回まで
グラフ	式 6 つ、steps 600（約 600 点。libqalculate が点を増減する）、再計算の待ち 350 ms、目盛り線 400 本まで
3D グラフ	式 4 つ、格子 25×25・41×41（初期値）・61×61、網目の線は 45×45 以下のとき、回転 0.01 rad/px（仰角 ±1.5 rad）、拡大 0.4〜6 倍、タップの範囲 20 px、z の外れ値の判定は値が 50 個以上のとき
入力欄の高さ	110 px × 文字の大きさの倍率（標準 1・大きめ 1.18・特大 1.38）まで
通知（トースト）	標準 2.2 秒。「元に戻す」ボタン付きは、行の削除 5 秒、履歴の消去 6 秒
行列の表表示	2 行以上、要素 400 個まで。結果が 20,000 文字以下のとき。「[」は 200 個まで調べる。文字の縮小は 60 % まで
一覧（関数・単位・変数）	表示は先頭の 400 件まで。検索は入力が止まって 150 ms 後

項目	値
元に戻せる式	1つ（次に入力・削除・確定するまで）
履歴の書き出し（CSV）	ファイル名 QCalc-history-年月日.csv、列は日時・式・結果・数値（3.23）

3.14 エラーコードとメッセージの一覧

Java が作る応答（code 付き）

どれも {"id":…,"ok":false,"error":"…","code":"…"} の形です。error の文は英語（版 1.3.0 から。それまでは日本語）で、日本語の画面では画面が MESSAGES_JA で日本語にして表示します。

code	error の文（英語）	日本語の画面での表示	起きるとき
bad_request	The request is not valid	要求の形式が正しくありません	要求に改行が含まれる、または JSON として読めない
timeout	The calculation took too long and was stopped	時間がかかりすぎたため計算を中止しました	待ち時間内に応答がない（エンジンを止め、次の要求で起動し直す）
cancelled	The calculation was stopped	計算を中止しました	利用者が中止した（エンジンを止めた）
crash	The calculation engine stopped unexpectedly. Try a simpler expression.	計算エンジンが異常終了しました。式を簡単にしてお試しください	要求を送ったあとにエンジンが終了した（同じ要求は送り直さない）
engine	Communication with the calculation engine failed (理由)	計算エンジンとのやりとりに失敗しました（理由）	送ったあとの読み書きに失敗した、または Java の処理中に想定外の例外が起きた
engine	Could not start the calculation engine (理由)	計算エンジンを起動できませんでした（理由）	起動または送信が 2 回続けて失敗した

エンジンが返すエラー（ok=false）

要求	error の文（英語）	日本語の画面での表示	意味
すべて	bad request（id は null）	英語のまま	JSON として読めない、オブジェクトでない、後ろに余分な文字がある
すべて	unknown command	英語のまま	cmd が分からない
すべて	engine error: 〈理由〉（restart:true）	英語のまま	処理中に例外が起きた。応答のあとエンジンは終了する
すべて	Not enough memory for the calculation（restart:true）	計算に必要なメモリが不足しました	計算スレッドなどで処理できない例外が起きた（終了コード 5）
calc、set_ans、define	The expression is nested too deeply	式の入れ子が深すぎます	3.13 の入れ子の上限を超えた
set_ans	expr missing	表示しない（画面と Java が自分で送る要求）	expr がない（計算できないときは ok=true、set=false）
plot	exprs missing / too many expressions / invalid range	invalid range は「x の範囲を確認してください」、ほかは英語のまま	式の配列がない / 12 個を超えた / 範囲が数にならない、または min ≥ max
plot3d	exprs missing / too many expressions / invalid x range / invalid y range	invalid x range は「x の範囲を確認してください」、invalid y range は「y の範囲を確認してください」、ほかは英語のまま	式の配列がないか空 / 4 個を超えた / x または y の範囲が数にならない、または最小 ≧ 最大
catalog	unknown catalog kind	英語のまま	kind が functions・variables・units のどれでもない
define_var、define_func	Enter a name and a value	名前と内容を入力してください	名前か内容が空

要求	error の文（英語）	日本語の画面での表示	意味
define_var、define_func	Check the value (libqalculate のメッセージ)	内容を確認してください（libqalculate のメッセージ。よく出るものは日本語）	内容を式として読めない（エラーか警告が出た）
define_var	The variable name contains characters that cannot be used	変数名に使えない文字が含まれています	libqalculate の変数名の規則に合わない
define_func	The function name contains characters that cannot be used	関数名に使えない文字が含まれています	libqalculate の関数名の規則に合わない
define_var、define_func	"名前" is a built-in name and cannot be used	「名前」は組み込みの名前なので使えません	組み込みの変数・単位・関数と同じ名前
define_func	"to" cannot be used in the expression of a function	関数の式に「to」は使えません	式に変換（to）が含まれる
delete_def	There is no user definition "名前"	ユーザー定義の「名前」が見つかりません	その名前のユーザー定義がない

時間切れは失敗ではなく ok=true、timeout=true の応答です（監視スレッドが答えたときは restart=true も付きます）。定義の保存に失敗したときも ok=true で、saved=false になります（画面は「設定しましたが、ファイルに保存できませんでした」と表示）。

為替レートの更新（RatesUpdater の応答）

状況	応答
1つ以上更新できた	ok=true、updated（更新できた数）、total（取得元の数）、failed（取得できなかった取得元のホスト名）、time
1つも取得できなかった	ok=false、error は Could not download the exchange rates. Check the network connection.（日本語の画面では「為替レートをダウンロードできませんでした。通信状態を確認してください。」）

状況	応答
エンジンが新しいファイルを読み込めなかったなど	ok=false、error は Could not update the exchange rates (理由)（日本語の画面では「為替レートの更新に失敗しました（理由）」。理由が The calculation engine could not load the new rates なら「計算エンジンが新しいレートを読み込めませんでした」）

画面が出すメッセージ

画面の文は英語と日本語の 2 通りあり、どちらも tr(英語, 日本語) で選びます（3.21）。次の表は日本語の画面の文です（英語の文は英語版の詳細設計書の 3.14）。

メッセージ	出るとき
計算エンジンを準備中です	起動の途中で＝を押した
（エラーの文）（ここをタップすると、もう一度試します）	init が失敗した。タップで起動をやり直す
計算エンジンから応答がありません	画面の最後の安全策の時間を過ぎた（code=timeout）
計算エンジンの応答を読み取れませんでした	応答が JSON として読めない（いちばん古い待ちの要求を失敗にする）
式を確認してください	結果が空でエラーメッセージもない
この結果はそのまま挿入できません（ans キーで使えます）	長すぎて短くした結果などをタップした
x の範囲を確認してください / グラフを計算できませんでした	グラフの範囲が正しくない / それ以外の失敗
y の範囲を確認してください	3D グラフの y の範囲が正しくない
時間がかかりすぎたため、途中までしか計算できませんでした	3D グラフの計算が制限時間内に終わらなかった（計算できた行までを描く）
実数の値がありません（x と y の範囲を確認してください）	3D グラフの値がすべて複素数・未定義など
関数を入力してください	3D グラフの式が空のまま「描画」を押した
「描画」を押すとグラフを描きます	前回のグラフの描画が最後まで終わらなかった（自動では描かない）
ライセンス文を読み込めませんでした。	licenses.js を読み込めなかった（3.16）

メッセージ	出るとき
これ以上は拡大・縮小できません	グラフの表示範囲が限界（3.10）
保存されていたユーザー定義のファイルが読めなかったため、別の場所に移しました	init に defs_error があった
画面の表示に必要な「Android System WebView」を利用できません。…	WebView を作れない（MainActivity）
画面の表示が繰り返し停止しました。アプリを閉じて、もう一度開いてください。	画面の表示処理が 15 秒以内に 2 回止まった（MainActivity）
1 行を削除しました / 履歴を消去しました（どちらも「元に戻す」ボタン付き）	履歴の行を削除した / 履歴を消去した（3.9）
履歴はありません	履歴が空のときにメニューの「履歴を消去」または「履歴を書き出す（CSV）」を選んだ
一覧を読み込めませんでした（理由）	関数・単位・変数の一覧を取得できなかった（「もう一度読み込む」ボタンを出す）
先頭の 400 件を表示しています。検索で絞り込んでください。	一覧の該当が 400 件を超えた
長い結果のため、途中を省略しています（全 N 桁）／（全 N 文字）	結果が 450,000 バイトを超えた（プレビューと履歴。3.9）
「…は QCalc Pro の機能です」（1000 桁を超える有効桁数、3D グラフ、配色テーマ、履歴の書き出し）と「QCalc Pro について」ボタン	Pro が使えないときに Pro の機能を使おうとした（3.20）
QCalc Pro が使えるようになりました。ありがとうございます！	購入などで Pro が使えるようになった
購入を確認しています…	Pro の画面で「購入を復元」を押した
この端末では Google Play を利用できません／Google Play に接続できませんでした。しばらくしてから、もう一度お試しください。／現在 QCalc Pro を購入できません／QCalc Pro の購入が見つかりませんでした／購入を完了できませんでした（コード）／支払いの手続き中です。完了すると QCalc Pro が使えるようになります。	PlayBilling が pro イベントの message で知らせた（英語の文を日本語にして表示。3.20）
履歴を書き出しました／ファイルを保存できませんでした	履歴の書き出しが終わった／失敗した（3.23。保存先を選ばずに閉じたときは何も出さない）

メッセージ	出るとき
コピーしました（Android 12 以前）／長すぎてコピーできませんでした	クリップボードに入れた／クリップボードが受け取らなかった（MainActivity。3.2）

libqalculate のメッセージの日本語化（MESSAGES_JA）

libqalculate のメッセージは英語です（日本語の翻訳はない）。日本語の画面では、MESSAGES_JA（57 個）でよく出る文を日本語にし、それ以外は英語のまま表示します。57 個は、次の表の libqalculate の文 28 個（解析と行列の 14 個は版 1.1.0 で追加）と、Java とエンジンの文 19 個（上の表）、Google Play の文 6 個（3.20）、ファイルの保存 1 個（3.23）、長い結果の注意 3 個（3.9）です。英語の画面では、どれも英語のまま表示します。プレビューでは最初のエラー（なければ最初の警告）を 1 つ、確定した結果には info を除いて 3 つまでを表示します。

英語（libqalculate）	日本語の表示
It has been N days since the exchange rates last were updated.	為替レートが N 日間更新されていません（メニュー→「為替レートの更新」）
Misplaced operator(s) "X" ignored. / Misplaced 'X' ignored.	余分な演算子「X」を無視しました / 余分な「X」を無視しました
Division by zero.	ゼロで割っています
Definite integral was approximated.	定積分は数値計算による近似値です
Empty expression in parentheses interpreted as zero.	空のかっこは 0 として計算しました
A global unit or variable was deactivated. …	同じ名前の組み込みの単位・変数を一時的に置き換えました（新しい変数を削除すると元に戻ります）
You need at least N argument(s) (A) in function F().	F() には引数が N 個以上必要です（A）
Did you mean "A" (instead of "B")?	「B」ではなく「A」のことですか？
limit() is incomplete and unreliable.	limit() は完全ではなく、結果が正しくない場合があります
No unknown variable/symbol was found.	解く対象の未知数（変数）が見つかりません
Conversion of only part of an expression is not supported.	式の一部だけを変換することはできません
Unable to find a solution.	解が見つかりませんでした

英語（libqalculate）	日本語の表示
Unable to integrate the expression. / Unable to integrate the expression exactly.	この式は積分できませんでした / この式は厳密には積分できませんでした
Unable to find limit.	極限を求められませんでした
No differential equation found.	微分方程式が見つかりません（例：dsolve(diff(y, x) = y)）
Unable to solve differential equation.	この微分方程式は解けませんでした
To avoid division by zero, the following must be true: C.	ゼロで割らないための条件：C
Was unable to isolate X with the current assumptions. The assumed (type and) sign was therefore temporarily set as unknown.	X を実数として求められなかったため、その仮定を外して解きました（複素数の解を含むことがあります）
Was unable to completely isolate X. / Was unable to isolate X.	X について完全には解けませんでした / X について解けませんでした
Inverse of singular matrix.	逆行列がありません（行列式が 0 の行列です）
Please use the cross(), dot(), and hadamard() functions for vector multiplication.	ベクトルどうしの積は、cross()（外積）、dot()（内積）、hadamard()（要素ごとの積）を使ってください
The second matrix must have as many rows (was R) as the first has columns (was C) for matrix multiplication.	行列の積は、左の行列の列の数（C）と右の行列の行の数（R）が同じでなければ計算できません
Argument N, A, in F() must be a square matrix.	F() の N 番目の引数は正方行列（行と列の数が同じ行列）でなければなりません
F() requires that all matrices/vectors have the same dimensions.	F() に渡す行列・ベクトルは、すべて同じ大きさでなければなりません

3.15 ウィンドウと戻る操作（Android 13〜16 への対応）

QCalc は targetSdk 36 なので、Android 15 以降では画面がシステムバーの裏まで広がり、Android 16 では従来の onBackPressed() が呼ばれません。MainActivity の setUpWindow() がこの 2 つを、古い版の動きを変えずに扱います。

画面の入れ物（setUpWindow）

- 根の FrameLayout（ナビゲーションバーと同じ色）の上に、ステータスバーの裏を塗る View（statusScrim）と、ページを入れる FrameLayout（content）を重ねる。WebView や案内の文は content に入れる。
- Android 15 以降（API 35〜）：根の View に OnApplyWindowInsetsListener を付ける。systemBars と displayCutout の insets を content の余白にし、下の余白はナビゲーションバーとキーボード（ime）の大きいほうにする。statusScrim の高さはステータスバーの高さ。insets はここで使い切る（CONSUMED）。
- Android 14 以前：insets の処理は付けず、システムの通常の配置に任せる（windowSoftInputMode="adjustResize" でキーボードの分だけ縮む）。

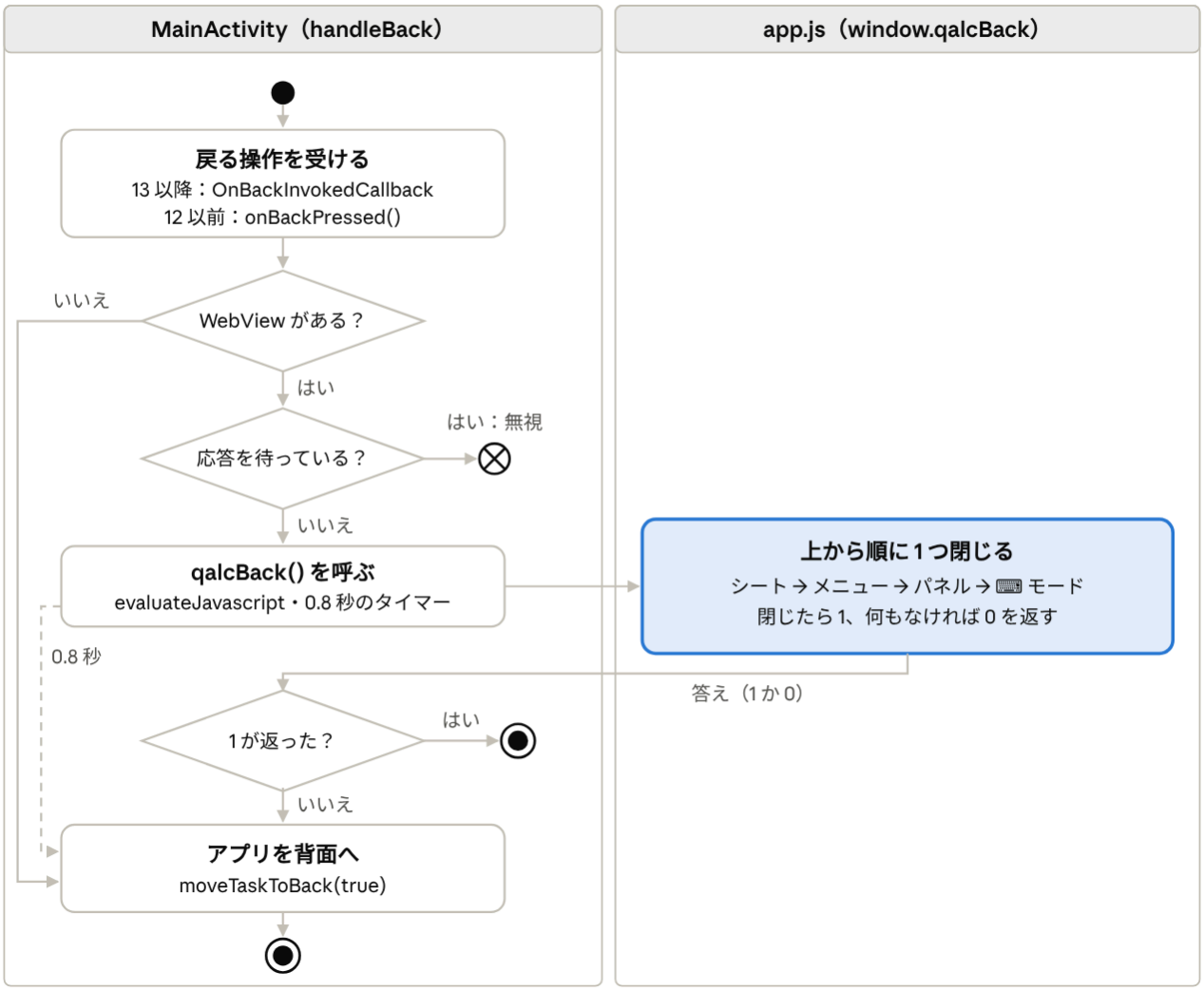
システムバーの色（applySystemBarColors）

Android	ステータスバー	ナビゲーションバー	アイコンの明暗
15 以降（API 35〜）	透明。裏の statusScrim の色（ライト #FFFFFF、ダーク #1C1F26。上部バーと同じ）	透明。根の View の色（ライト #F6F7F9、ダーク #121418）。setNavigationBarContrastEnforced(false) で半透明の幕を付けない	WindowInsetsController
11〜14（API 30〜34）	setStatusBarColor（同じ色）	setNavigationBarColor（同じ色）	WindowInsetsController
8.0〜10（API 26〜29）	setStatusBarColor	setNavigationBarColor	SYSTEM_UI_FLAG_LIGHT_STATUS_BAR・LIGHT_NAVIGATION_BAR
7.0・7.1（API 24・25）	setStatusBarColor	setNavigationBarColor（明るいバーのアイコンがないため、ライトでも #121418）	SYSTEM_UI_FLAG_LIGHT_STATUS_BAR

ダークモードを切り替えると Activity が作り直され（configChanges に uiMode を含めていない）、色を選び直します。

戻る操作（handleBack）

戻る操作は上に重なったものを 1 つ閉じ、何もなければアプリを背面に回す



アクティビティ図·戻る操作の判定

- Android 13 以降（API 33〜）は OnBackInvokedDispatcher に OnBackInvokedCallback（PRIORITY_DEFAULT）を登録し、マニフェストで android:enableOnBackInvokedCallback="true" にする。Android 12 以前は onBackPressed() を使う。どちらも handleBack() を呼ぶ。
- handleBack() は画面の window.qalcBack() を evaluateJavascript で呼ぶ。画面は上に重なっているもの（シート → メニュー → パネル → キーボード入力モード）を 1 つ閉じて 1 を、何もなければ 0 を返す。
- 0 のときは moveTaskToBack(true) でアプリを背面に回す（プロセスと入力中の式は残る。Android 12 以降のシステムの既定と同じ動き）。
- 0.8 秒たっても答えがなければ（画面が処理中）背面に回す。答えを待つあいだの 2 回目の戻る操作は無視する（backPending）。WebView がないとき（作れなかった、表示処理が止まった）はすぐに背面に回す。
- コールバックを常に登録しているので、予測型「戻る」の「ホームに戻る」アニメーションは出ず、指を離したときに上の動きになる。

3.16 オープンソースライセンスの表示

アプリ自身（QCalc）と、組み込んだ 7 つのソフトウェアの著作権表示とライセンス全文（計 8 件）を、「このアプリについて」→「オープンソースライセンス」で表示します。文は make-licenses.py が元のソースのライセンスファイルから app/assets/licenses.js（約 90 KB）にまとめます。

- licenses.js は window.QCALC_LICENSES に、components（部品ごとの name・version・license・copyright・url・texts）、titles（ライセンス文の名前）、texts（全文。同じ文は 1 回だけ）を入れる。
- パネルを開いたときに初めて script 要素で読み込む（起動を遅くしないため）。読めなければ「ライセンス文を読み込めませんでした。」と表示する。
- 部品ごとにカードを作り、ライセンス文は details 要素に入れて、開いたときに初めて文字を入れる（長い文を最初から並べないため）。文字は textContent で入れ、HTML として解釈しない。
- ライブラリの版を変えたときは、fetch-sources.sh のあとで make-licenses.py を実行し直す（ビルド・保守マニュアル 5.9）。

部品	版	ライセンス	著作権表示	収録する全文（読み取り元）
QCalc	1.3.0（ライセンスの画面には版を出さない。版は「このアプリについて」に表示）	GNU GPL v2 以降	Copyright (C) 2026 HiroshiTsu	GPL v2
libqalculate	5.13.2	GNU GPL v2 以降	Copyright (C) 2003-2007, 2008, 2016-2024 Hanna Knutsson	GPL v2（libqalculate の COPYING）
GNU MP（GMP）	6.3.0	GNU LGPL v3 以降、または GNU GPL v2 以降	Copyright 1991, 1993-1997, 1999-2016, 2020, 2021 Free Software Foundation, Inc.	LGPL v3、GPL v3、GPL v2

部品	版	ライセンス	著作権表示	収録する全文（読み取り元）
GNU MPFR	4.2.2	GNU LGPL v3 以降	Copyright 1999-2025 Free Software Foundation, Inc.	LGPL v3、GPL v3（MPFR の COPYING.LESSER・COPYING）
libxml2	2.15.4	MIT	Copyright (C) 1998-2012 Daniel Veillard. Copyright (C) The Libxml2 Contributors.	libxml2 の Copyright
musl libc	Zig 0.17.1 に同梱	MIT	Copyright © 2005-2020 Rich Felker, et al.	musl の COPYRIGHT（zig パッケージ）
LLVM libc++ / libc++abi / libunwind	Zig 0.17.1 に同梱	Apache License v2.0 with LLVM Exceptions	LLVM Project contributors	libcxx の LICENSE.TXT（zig パッケージ）
Zig compiler_rt	0.17.1	MIT	Copyright (c) Zig contributors	Zig の LICENSE

MPFR が LGPL v3 以降のため、計算エンジン（libqalcengine.so）全体は GNU GPL v3 以降の条件で配布します（GPL v2 以降の libqalculate と一緒にできるのは v3）。対応するソースはリポジトリの third_party/ に置き、アプリの「QCalc のソースコード（GitHub）」から辿れるようにします。

QCalc 自身のコード（third_party/ 以外）には、Google Play Billing Library とそれが使うライブラリにリンクして配布してよいという例外を付けています（リポジトリの README の License。版 1.3.0）。libqalculate などほかの GPL のソフトウェアにはこの例外は及びませんが、アプリのコードは libqalculate にリンクせず、別のプログラム（計算エンジン）として動かしています。

3.17 App Bundle の構成と署名

Google Play に出す App Bundle は、Gradle や bundletool を使わず build-aab.sh が組み立てます。できた .aab は bundletool 1.17.2 の validate で検査し、端末向けの APK を作れることも確かめています（6 章）。

パス	中身	作り方
BundleConfig.pb	bundletool の版（1.17.2）と、「ネイティブライブラリを圧縮しない」最適化の無効化	build-aab.sh の Python が protobuf を直接書く
base/manifest/AndroidManifest.xml	package io.github.hiroshitsu.qcalc、versionCode 5、versionName 1.3.0、minSdk 24、targetSdk 36（protobuf 形式）	aapt2 link --proto-format
base/resources.pb、base/res/	アプリ名・テーマ・アイコン（protobuf 形式のリソース表）	aapt2 compile・link（API 34 の android.jar）
base/assets/	画面（index.html・style.css・app.js・licenses.js）	そのまま
base/dex/classes.dex	Java の 5 ファイル（app/src。PlayBilling は含まない）をコンパイルしたもの	javac（API 36 の android.jar、Java 8）→ dalvik-exchange
base/lib/arm64-v8a/libqalcengine.so	計算エンジン（7,588,488 バイト、静的リンクの実行ファイル）	build-engine.sh
META-INF/	アップロード鍵の署名（SHA-256 with RSA）	jarsigner

- aapt2 は、Ubuntu の版が API 35 以降の android.jar のリソース表を読めないため、API 34 の android.jar でリンクする（使うリソースは API 34 までのもの）。Java は API 36 の android.jar でコンパイルし、OnBackInvokedCallback や WindowInsets などの新しい API をそのまま参照する。
- BundleConfig の uncompress_native_libraries を enabled: false にする。bundletool の既定では、作る APK に .so を無圧縮で入れて extractNativeLibs を false にするため、エンジンがファイルとして展開されず実行できない。無効にすると APK の中のエンジンは圧縮され（7.6 MB → 約 3.1 MB）、インストール時に nativeLibraryDir へ展開される。
- 16 KB ページ：エンジンの読み込み区画（LOAD）4 つはすべて 0x4000（16 KB）境界に揃えてあり、16 KB ページの端末でも動く。Google Play は Android 15 以降を対象とするアプリに 16 KB ページへの対応を求めている。

- 署名：jarsigner（-digestalg SHA-256、-sigalg SHA256withRSA）でアップロード鍵 signing/qcalc-upload.jks（別名 upload）の署名を付け、jarsigner -verify で確かめる。パスワードは環境変数 QCALC_UPLOAD_PASS か signing/qcalc-upload.pass の 1 行目から読む。Google Play はこの署名を確かめたうえで、アプリ署名鍵で署名した APK を配る。
- 版：version.properties の VERSION_NAME と VERSION_CODE を build-apk.sh・build-aab.sh・Gradle が読む。Google Play に出すたびに VERSION_CODE を増やす。
- ZIP の各項目の時刻は 1981-01-01 に固定し、同じ入力からは署名を除いて同じ中身になるようにしている。
- QCalc Pro の購入に使う PlayBilling は Google Play Billing Library が要るため、build-aab.sh の App Bundle には入らない（billing は none になり、Pro の機能をすべて使える）。購入できる App Bundle は Gradle で qcalc.billing=true にして作る（ビルド・保守マニュアル。3.20）。

3.18 式の例、一覧と日本語の検索

版 1.1.0 で、式の書き方を覚えていなくても使えるように、式の例のパネルを加え、関数・単位・変数の一覧を日本語で探せるようにしました。版 1.3.0 で画面を英語と日本語の 2 通りにしたため、式の例の説明と一覧の分類名は表示の言語で出し、日本語の名前と日本語での検索は日本語の画面だけで使います。

式の例・使い方（openExamples、useExample）

- EXAMPLES は「分野の名前と、[式, 説明] の並び」の配列で、12 分野・69 個（基本の計算 9、単位の変換 8、通貨の換算 4、方程式 4、微分 6、積分 6、極限・級数・微分方程式 7、行列 8、ベクトル 4、進数・整数 5、日付 3、統計・誤差・定数 5）。TIPS は使い方のヒント 11 個。
- 例をタップすると、開いているパネルをすべて閉じて、式を入力欄に入れる（useExample → setExpr）。入力中だった式は「元に戻す」で戻せる。計算はプレビューが行い、= を押すまで履歴には残らない。
- 例の式は、画面のキーが入れる文字（×、−、π）で書いてある。すべての例と最初の画面の 4 つは、テストでエンジンに通し、エラーなく結果が出ることを確かめている（73 式。テスト仕様書・結果）。英語の画面では通貨の例の 3 式が違い（100 USD to EUR、50 EUR to GBP、1 BTC to USD。最初の画面の例も 100 USD to EUR）、この 3 式もエンジンに通してエラーなく結果が出ることを確かめた（版 1.3.0）。
- 開く場所は、メニューの「式の例・使い方」、最初の画面のボタン、「このアプリについて」のリンクの 3 つ。

一覧の読み込み（loadCatalog）

- catalog コマンドで関数・変数・単位を取得し、種類ごとに覚えておく。ユーザー定義を変えたときと為替レートを更新したときは捨てて、次に開いたときに取り直す（invalidateCatalogs）。
- このアプリでは使えないもの（HIDDEN_ITEMS：関数 plot・command・load・export、変数 uptime）を除く。関数は、エンジンの 425 個のうち 421 個を表示する。
- 日本語の画面では、項目ごとに日本語の名前と分類の日本語名（CATEGORY_JA）を付ける（英語の画面では付けず、英語の名前と分類で表示・検索する）。日本語の名前は、関数が FUNC_JA（230 個）、変数・定数が VAR_JA（61 個）、単位・通貨が JA_NAMES（124 個。単位の名前か abbr で引く）。
- 検索用の文字列は、名前・abbr・表示名・英語の題・説明・分類・日本語の名前（日本語の画面だけ）をつなげて小文字にしたもの。
- 表（FUNC_JA など）は外から来る名前（利用者が付けた変数名など）で引くので、Object.prototype から受け継いだ値（constructor など）を拾わないように own() を通す。

検索と並び順（openCatalog）

- 検索欄の文字（小文字にして前後の空白を除く）を含む項目だけを表示する。分類のボタン（「すべて」と最上位の分類）でも絞り込める。単位の一覧は、最初は「通貨」（Currency）を表示する。
- 検索しているときは、合い方の良い順に並べる。① 名前・abbr・日本語の名前のどれかと完全に同じ、② 名前か題が検索語で始まる、または日本語の名前に含まれる、③ 名前か題に含まれる、④ 説明や分類にだけ含まれる。同じ順位の中は題の順。たとえば関数の一覧では、「積分」で Integrate（integrate）が、「min」で Min（最小値）が先頭に来る。
- 表示は先頭の 400 件まで。該当の数を「n 件」と表示する。
- openCatalog(種類, 分類) で分類を指定すると、検索を空にし、その分類を選んだ状態で開く（「解析」の「ほか…」は微積分、「行列」の「ほか…」は行列・ベクトル）。横に並ぶ分類のボタンは、選んだものが見える位置までスクロールする。

行の表示と挿入（catalogRow、useItem）

一覧	表示	ボタン
関数	題と日本語の名前、書き方（syntax）、分類、説明	「挿入」:「名前()」を入れてカーソルを括弧の中に置く
単位・通貨	題と日本語の名前、式に書く文字（abbr。内部の名前と違うときは括弧で併記。複合単位は内部の名前を出さない）、分類、説明	「変換」:「 to 文字」を入れる。「挿入」：文字を入れる（insert=false の単位には出さない）

一覧	表示	ボタン
変数・定数	題と日本語の名前、名前（表示名と値）、分類、説明	「挿入」：名前を入れる

どのボタンも、開いているパネルをすべて閉じてから、入力欄のカーソル位置に入れます。行をタップすると説明の全文を開閉します。

単位を式に書く文字（エンジンの unit_input_text、reads_back_as）

- 複合単位（km/h、kW·h など）は、libqalculate の中では km_p_h、kW_h のような名前で、その名前を式に書いても単位として読まれない。そこで一覧には、結果を表示するときの形（短い名前、Unicode の記号、接頭辞付き）を使う。それ以外の単位は、入力に向いた名前（preferredInputName）を使う。
- その文字が本当にその単位として読めるかを確かめる。「3 文字」を計算した結果と、3 × その単位を、どちらも基本単位に直して比べる（たとえば Ha/e は「ハートリーをネイピア数 e で割ったもの」と読まれるので不合格）。不合格なら内部の名前でも試し、それも読めなければ insert=false を返す（変換先としては使える）。
- 通貨は確かめない。名前が通貨コードか記号だけであり、確かめるための計算が「為替レートが古い」の警告（1 回だけ出る）を先に使ってしまうため。
- 合格した文字は覚えておく。不合格は、次に一覧を取得するときにもう一度試す（利用者の変数 m = 3 が単位 m を隠していた、という場合があるため）。

3.19 計算エンジンへの追加と libqalculate の問題への対処

libqalculate のソースは変えていません。次の 3 つは、橋渡しプログラム（qalc-engine.cc）が、動いている計算エンジンに対して行います。単位と関数の追加は、利用者の定義ファイルには保存されず、エンジンを起動するたびに行います。それ以外の計算の結果は、公式の qalc と同じです（309 式で比較。テスト仕様書・結果）。

日本の単位（add_japanese_units）

単位	読み	種類	定義（厳密な値）	およそ
坪	つぼ（tsubo とも書ける）	面積	400/121 m²（1 間 × 1 間）	3.3058 m²
反	たん	面積	120000/121 m²（300 坪）	991.74 m²
寸	すん	長さ	1/33 m（1/10 尺）	3.0303 cm
尺	しゃく	長さ	10/33 m	30.303 cm
間	けん	長さ	20/11 m（6 尺）	1.8182 m
里	り	長さ	43200/11 m（2160 間）	3.9273 km

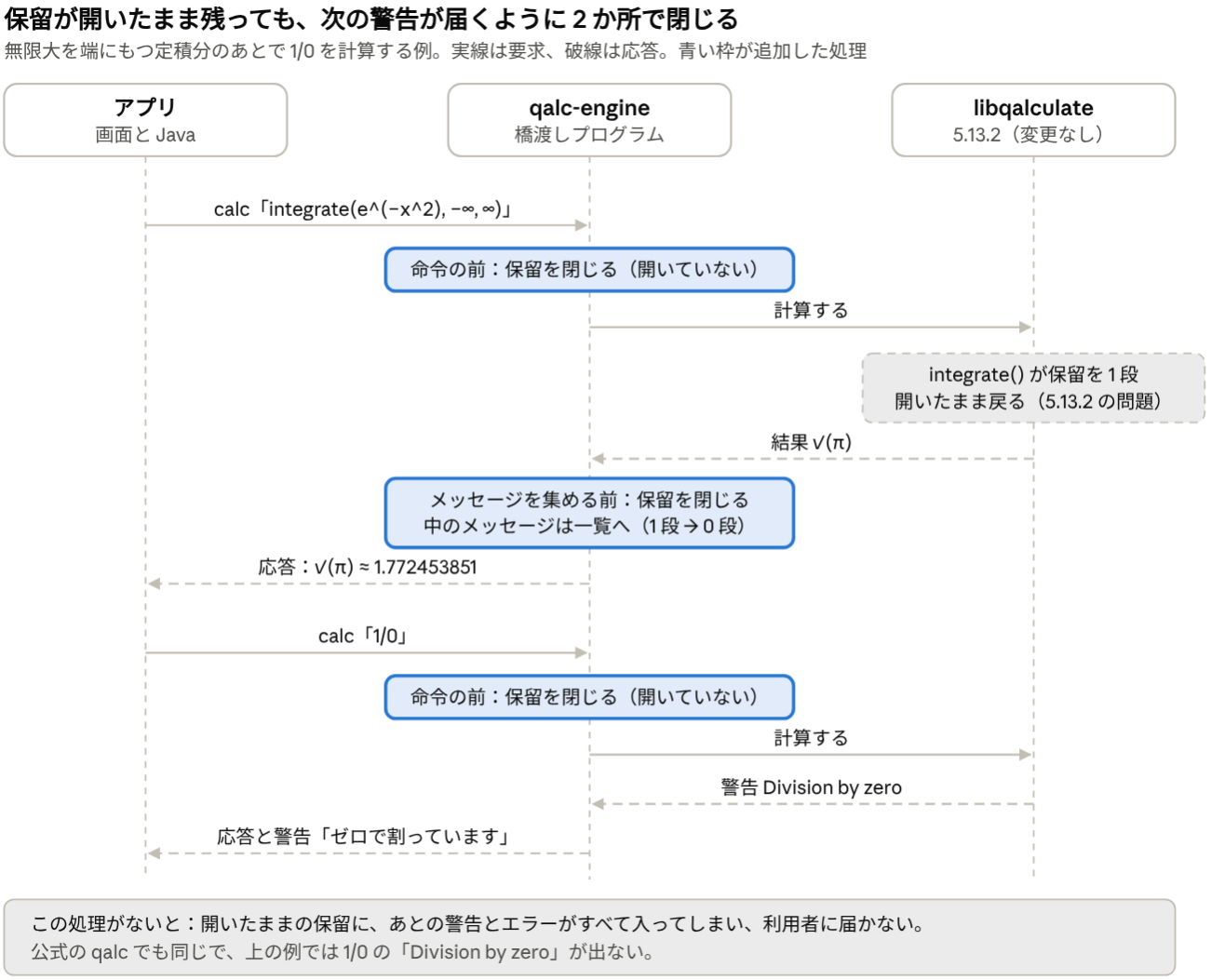
単位	読み	種類	定義（厳密な値）	およそ
合	ごう	体積	2401/13310 L（1/10 升）	180.39 mL
升	しょう	体積	2401/1331 L	1.8039 L
匁	もんめ	質量	15/4 g	3.75 g
貫	かん	質量	3750 g（1000 匁）	3.75 kg

- 値は 1891 年の度量衡法の定義（1 尺 = 10/33 m、1 升 = 2401/1331 L、1 貫 = 3.75 kg）による厳密な分数で持つ（例：30 坪 to m² ≈ 99.17355372 m²）。
- libqalculate の AliasUnit として、分類 Japanese Units（一覧では「日本の単位」）に追加する。ラテン文字の名前は坪の tsubo だけ。ken・go・tan のような短い名前は、式の中で別の意味に読まれるおそれがあるので付けない。
- 同じ名前の単位や変数がすでにあるときは追加しない（libqalculate の将来の版が同じ単位を持った場合など）。

行列のトレース trace()（TraceFunction）

- libqalculate には det()（行列式）・adj()（余因子行列）・rk()（階数）・cofactor() などがあるが、トレース（対角成分の和）がない。引数 1 つ（正方行列を要求する MatrixArgument）の関数 trace を、分類 Matrices & Vectors に追加する。
- 計算は、対角成分 m[i][i] を順に足す（calculateAdd）。中止が要求されたら途中でやめる。正方行列でない引数は、libqalculate が「正方行列でなければなりません」のエラーにする。
- 例：trace([1, 2; 3, 4]) = 5。文字を含む行列でも計算できる（trace([x, 1; 2, y]) = x + y）。
- libqalculate にない行列の計算のうち、固有値は既存の関数で求められる（solve(det(M − x × identity(n)) = 0)。式の例に収録）。rank() は順位を返す別の関数で、行列の階数は rk()。

警告・エラーが出なくなる問題への対処（close_message_stops）



シーケンス図 · メッセージの保留の閉じ忘れと対処

- 問題：libqalculate は、計算の途中で出る警告やエラーを一時的にためておく「メッセージの保留」（beginTemporaryStopMessages で開き、endTemporaryStopMessages で閉じる）を、入れ子にして使う。5.13.2 の MathStructure::integrate() には、無限大を端にもつ定積分が解けたときに、保留を閉じないまま戻る経路がある。保留が 1 段開いたままになると、そのあとの警告とエラーはすべて保留の中に入り、利用者に届かない。公式の qalc でも同じで、integrate(e^(−x^2), −∞, ∞) のあとに 1/0 を計算しても「Division by zero」が出ない。
- 対処：保留は、命令と命令の間や、メッセージを集める時点では、開いていないはずのもの。そこで橋渡しプログラムが次の 2 か所で、開いている保留をすべて閉じる（endTemporaryStopMessages が「開いている保留はない」を返すまで繰り返す）。
- ① メッセージを集める前（collect_messages）：中身をメッセージ一覧に移す。いま終わった計算のメッセージなので、その計算の messages として返す。
- ② 命令を実行する前（main のループ）：中身は捨てる（前の命令の残りなので、次の命令のメッセージに混ぜない）。

- libqalculate の将来の版でこの問題が直れば、開いている保留がないので、この処理は何もしない。テスト M03〜M05 で確かめている（テスト仕様書・結果）。

解析・行列で知っておく libqalculate の性質

性質	アプリでの扱い
limit() は必ず「limit() is incomplete and unreliable.」の警告を出す	日本語にして表示する（結果は出る）
dsolve() が解けるのは 1 階の微分方程式	キーのひな形を dsolve(diff(y, x) =) にし、解けないときのメッセージを日本語にする
無限級数 sum(…, 1, ∞) は計算されない	キーのひな形は sum(, 1, 10)。式の例は有限の和と積だけ
1 文字の名前（a・b・m・t など）は単位として読まれる	解析のキーと式の例では変数に x・y・z・n を使う
行列は [1, 2; 3, 4]、[1 2; 3 4]、[[1, 2], [3, 4]] のどれでも入力でき、1 行の文字 [1 2; 3 4] で表示される	「行列」のキーは [ ] ; を入れ、結果は画面が表の形にする（3.9）

3.20 QCalc Pro（版 1.3.0）

QCalc Pro は Google Play の買い切りのアイテム（商品 ID qcalc_pro）で、有効桁数 1,000 桁を超える計算、3D グラフ、配色テーマ、履歴の書き出し（CSV）と、今後の版で加える Pro の機能を使えるようにします。そのほかの機能はすべて無料のまま使えます。購入の状態は Java（Pro と PlayBilling）が持ち、画面（app.js）は Pro の機能を使う前にその状態を確かめます。

部品の分担

部品	役割
Pro（Pro.java）	状態（購入済み owned、支払い待ち pending、以前からの利用者 legacy、価格 price）を持ち、SharedPreferences「pro」に保存する。billing の種類を決め、info() で画面に返し、状態が変わると pro イベントを送る

部品	役割
PlayBilling（app/src-billing）	Google Play Billing Library 8 で Google Play に接続し、価格と購入を問い合わせ、購入画面を開き、購入を承認する。Pro.Store を実装し、結果を Pro.Owner で Pro に知らせる。Gradle で qcalc.billing=true にしたビルドだけに入り、Pro はこのクラスを名前で読み込む（ないビルドでも動く）
画面（app.js）	proInfo() の JSON を PRO に読み（readPro）、Pro の機能を使う前に hasPro() を確かめる。使えないときは案内（proPrompt）を出す。Pro の画面（openPro）で購入と復元を受け付ける

billing の種類

billing	ビルド	Pro の機能	画面
play	課金ありのビルド（PlayBilling が入っている）	購入したか、以前からの利用者なら使える	メニューに「QCalc Pro」を出し、使えないあいだは Pro の機能に PRO の印を付ける
debug	テスト版（パッケージ名が .debug で終わる）	最初から使える。Pro の画面の「Pro を有効にする（テスト用）」で外せる（setTestPro。test_locked に保存）	play と同じ。外したときは Pro なしの見え方になる
none	課金なしのビルド（build-apk.sh・build-aab.sh で作るもの）	買えないので、すべて使える	「QCalc Pro」の項目を出さず、何も制限しない

hasPro() は「billing が play でも debug でもない、または pro が true」のときに真です。画面は pro イベントのたびに proInfo() を読み直します（proChanged）。

Pro の機能を使うとき（画面）

機能	Pro が使えないとき
有効桁数 1,000 桁を超える計算	設定の欄の上限を 1,000 にし、超える数を入れると 1,000 に戻して案内を出す。エンジンに送る precision も 1,000 までに抑える（engineOpts）

機能	Pro が使えないとき
3D グラフ	3D のボタンを押すと案内を出して 2D のままにする。保存された種類が 3d でも 2D で開く
配色テーマ	ブルー以外を選ぶとブルーに戻して案内を出す。保存されたテーマがあってもブルーで表示する（applyTheme）
履歴の書き出し	メニューの「履歴を書き出す（CSV）」を選ぶと案内を出す

案内（proPrompt）は「…は QCalc Pro の機能です」のシートで、「QCalc Pro について」を押すと Pro の画面を開きます。Pro が使えないあいだは html 要素に pro-locked クラスを付け、Pro の機能の項目（data-pro 属性のある要素）に「PRO」の印を出します。Pro が使えるようになると「QCalc Pro が使えるようになりました。ありがとうございます！」を表示し、配色テーマと有効桁数の上限を反映して、入力中の式を計算し直します。

購入と確認の流れ（PlayBilling）

1. 起動の最初（MainActivity.onCreate）に Pro を作り、start で Google Play に接続して価格と購入を問い合わせる。画面に戻るたび（onResume）と Pro の画面を開いたときも問い合わせ直す。アプリの外での購入（Play ストアでのプロモーションコードの利用など）や、あとで完了した支払いを見つけ、払い戻された購入を数えないため。
2. 「QCalc Pro を購入」：価格の情報（ProductDetails）がまだなければ問い合わせてから、購入画面（launchBillingFlow）を開く。購入の選択肢（offer token）は、従来の形のものがあればそれ、なければ最初のものを使う。
3. 購入画面と問い合わせの結果は handle で調べる。qcalc_pro の購入が PURCHASED なら owned にし、まだ承認していなければ承認する（Google Play は 3 日以内に承認されない購入を払い戻すため。失敗したら次の問い合わせでもう一度）。PENDING なら支払い待ちにし、「支払いの手続き中です…」を知らせる。
4. 「購入を復元」：購入を問い合わせ直し、見つからなければ「QCalc Pro の購入が見つかりませんでした」を知らせる。
5. Pro は状態を SharedPreferences「pro」に保存し、pro イベント（{"event":"pro"}。知らせる文があれば message）を画面に送る。
6. 接続や問い合わせに失敗したときは、それまでの状態を保つ（購入済みなら使えるまま）。失敗の文を知らせるのは、利用者が操作したとき（購入・復元）だけ。

購入画面と支払いは Google Play が扱い、アプリは支払いの情報を受け取りません。
PlayBilling が知らせる文は英語で、日本語の画面では画面が日本語にします（3.14）。

以前からの利用者（legacy）

Pro が始まる前から QCalc を使っている人は、購入しなくても Pro の機能を使えます（LEGACY_USERS_KEEP_PRO = true）。

- 課金のないビルドを起動するたびに、印 pre_pro を付ける。
- 課金のあるビルドを初めて起動したとき（印 era がまだないとき）、pre_pro があるか、files/state に保存された状態があるか、WebView のデータのフォルダ（app_webview）があれば、legacy を付ける。この判定は、WebView やエンジンが何かを書く前（onCreate の最初）に行う。
- そのため、課金のないビルド（版 1.3.0 の App Bundle を含む）を使った人は、あとで課金のあるビルドに更新しても、Pro の機能を無料で使える。
- legacy は端末の中の印なので、アプリのデータを消すと戻らない。購入（owned）は Google アカウントに記録されるので、入れ直しても問い合わせで戻る。
- Pro の画面は、legacy のときは「QCalc Pro が始まる前から QCalc を使っていただいているので、無料で使えます。」と表示する。

配布物と billing

build-apk.sh と build-aab.sh は PlayBilling を入れないので、これらで作った版（版 1.3.0 の App Bundle を含む）は billing が none で、Pro の機能をすべて使えます。購入できる版は、Gradle で qcalc.billing=true にして作ります（Google Play Billing Library を Google の Maven リポジトリから取得する。ビルド・保守マニュアル）。

3.21 表示の言語（版 1.3.0）

画面は英語と日本語の 2 通りで、英語が標準です。利用者がメニューまたは設定の「Language / 言語」で日本語を選んだときだけ日本語にします（端末の言語設定には合わせません）。

- 言語の保存：選んだ言語を qalc.lang（files/state/qalc.lang.json。"en" または "ja"）に保存する（setLanguage）。設定（qalc.settings）の lang は使っている言語の控えで、起動時に qalc.lang の値に合わせる。言語を設定とは別に保存するのは、版 1.3.0 のテスト版がほかの設定といっしょに言語を保存していたため（利用者が選んでいない言語で始まらないように）。
- 起動時：app.js は最初に qalc.lang を読んで LANG（en・ja）と JA を決め、html 要素の lang 属性にする。言語は起動中には変えない。
- 文の選び方：画面の文は、英語と日本語を並べて書いた tr(英語, 日本語) で選ぶ。index.html の固定の文は英語で、日本語のときだけ起動時に置き換える（localizeStatic）。
- 言語を変えるとき：setLanguage が qalc.lang と設定を保存し、履歴と入力中の式をすぐ保存して（qalcPause）、ページを読み込み直す。履歴、入力中の式、ans、ユーザー定義はそのまま残る。設定を初期値に戻しても、言語は変えない。

- エンジンと Java の文：エンジン（libqalculate と qalc-engine.cc）と Java が画面に返す文は英語。日本語の画面では、画面が MESSAGES_JA（57 個）で日本語にする（localMessage）。表にない文は英語のまま表示する（3.14）。
- Java が自分で表示する文（WebView を使えないとき、表示処理が繰り返し止まったとき、コピーの通知）：MainActivity.tr(英語, 日本語) が qalc.lang を読んで選ぶ。
- 言語で変わる内容：最初の画面と式の例の通貨の式（英語では 100 USD to EUR など、日本語では円への換算）、上部のボタン（DEC・HEX など／10進・16進 など）、キーの読み上げの名前、CSV の見出し、プライバシーポリシーの開く位置（日本語では日本語の部分）。日本語の名前（関数・変数・単位）と日本語での検索、日本の単位の説明は、日本語の画面だけ（3.18）。
- 言語で変わらないもの：計算（ローカル通貨は端末の地域で決まり、日本では JPY）、式の書き方（関数名や単位は英語のまま）、全角文字などの変換（normalizeJa）。

3.22 配色テーマ（版 1.3.0）

キーと強調表示の色を 7 色から選べます（設定「配色テーマ」、theme）。QCalc Pro の機能です（3.20）。

テーマ（theme）	強調の色（ライト）	強調の色（ダーク）
ブルー（blue。既定）	#2563EB	#6B9CFF
グリーン（green）	#15803D	#4ADE80
ティール（teal）	#0F766E	#2DD4BF
パープル（purple）	#7C3AED	#A78BFA
ローズ（rose）	#BE123C	#FB7185
オレンジ（orange）	#C2410C	#FB923C
グラファイト（graphite）	#374151	#CBD5E1

- 色は style.css の CSS 変数で決める：--accent（強調の色）、--accent-text（強調の上の文字）、--accent-soft（薄い強調）、--key-op（演算子キーの背景）、--key-op-text（演算子キーの文字）。ブルーは style.css の既定の値で、ほかの 6 色は html 要素の data-theme 属性でこれらを上書きする。ライト（:root[data-theme=…]）とダーク（:root.dark[data-theme=…]）の両方に値がある。
- applyTheme は、ダークモードかどうか（appInfo() の dark、ないときは prefers-color-scheme）で dark クラスを付け、Pro が使えてテーマがブルー以外のときだけ data-theme を付ける。起動時、設定を変えたとき、Pro の状態が変わったときに呼ぶ。
- システムバーの色（3.15）はテーマでは変えない。

3.23 履歴の書き出し（CSV。版 1.3.0）

メニューの「履歴を書き出す（CSV）」で、履歴を CSV ファイルに保存します。QCalc Pro の機能です（3.20）。

1. exportHistory：Pro が使えなければ案内を出し、履歴が空なら「履歴はありません」と表示して終わる。
2. historyCsv で CSV を作る。文字コードは UTF-8 で、先頭に BOM を付ける（Excel が UTF-8 として読むため）。改行は CRLF。1 行目は見出し（Date・Expression・Result・Number。日本語の画面では日時・式・結果・数値）。
3. 各行は、日時（YYYY-MM-DD HH:MM）、式、表示どおりの結果（先頭の「= 」を除く。長い結果は「先頭 … 末尾」）、数値（10 進の数 1 つで表せる結果だけ。区切りの空白を除き、マイナスは -。比較・切った結果・10 進以外の結果は空）。どの欄も " で囲み、中の " は "" にする。式と結果が =、+、-、@、タブ、CR で始まるときは前に ' を付ける（表計算ソフトで式として実行されないため）。
4. ファイル名は QCalc-history-年月日.csv（例：QCalc-history-20261113.csv）。
5. アプリでは QalcNative.saveFile(名前, "text/csv", 内容) を呼ぶ。MainActivity は名前・種類・長さを確かめ（3.2）、システムのファイル選択画面（ACTION_CREATE_DOCUMENT）を開く。内容は、選んでいるあいだメモリに持つ（pendingSave）。
6. 場所が選ばれたら（onActivityResult）、qalc-save スレッドでその場所を "wt"（中身を置き換える）で開いて UTF-8 で書き、結果を saved イベントで知らせる（成功は ok、選ばずに閉じたら cancelled、失敗は error の Could not save the file）。選んでいるあいだにアプリが作り直されて内容がなくなったときも error にする。
7. 画面（fileSaved）は、成功なら「履歴を書き出しました」、失敗なら「ファイルを保存できませんでした」を表示する。選ばずに閉じたときは何も表示しない。
8. PC のブラウザで確かめるとき（QalcNative がない）は、同じ内容を Blob にしてダウンロードさせる。

アプリは保存した場所を覚えず、ファイルを読み直したり送ったりもしません。ファイルはアプリ専用の領域の外にあるので、アプリを消しても残ります。

例（英語の画面。テストで書き出したファイル）

```
"Date","Expression","Result","Number"
"2026-11-13 02:53","1/3","≈ 0.3333333333","0.3333333333"
"2026-11-13 02:53","-5 + 2","-3","-3"
"2026-11-13 02:53","'=1+1","false",""
"2026-11-13 02:53","2 m + 30 cm","2.3 m",""
```